Архитектура приложения

Содержание лекции



Компоненты приложения

Компоненты приложения являются основными строительными блоками приложения для Android. Каждый компонент - это точка входа, через которую система или пользователь могут войти в ваше приложение. Некоторые компоненты зависят от других.

Есть четыре разных типа компонентов приложения:

- Activities
- Services
- Broadcast receivers
- Content providers

Каждый тип служит определенной цели и имеет отдельный жизненный цикл, который определяет, как компонент создается и уничтожается. В следующих разделах описаны четыре типа компонентов приложения.

Компоненты приложения

Activities

Activity - это точка входа для взаимодействия с пользователем. Представляет собой единый экран с пользовательским интерфейсом. Например, приложение электронной почты может иметь одно activity, которое показывает список новых сообщений электронной почты, другое activity для составления сообщения электронной почты и еще одно activity для чтения сообщений электронной почты. Хотя все activity работают вместе, чтобы сформировать единый пользовательский интерфейс в приложении электронной почты, каждое из них не зависит от других. Таким образом, другое приложение может запускать любое из этих действий, если приложение электронной почты позволяет это. Например, приложение камеры может запустить activity в приложении электронной почты, которое составляет новое письмо, чтобы пользователь мог поделиться изображением.

Компоненты приложения

Activities

Activity обеспечивает следующие ключевые взаимодействия между системой и приложением:

- Отслеживание того, что в настоящее время волнует пользователя (что отображается на экране), чтобы гарантировать, что система продолжает выполнять процесс, в котором выполняется activity.

- Зная, что ранее использованные процессы содержат вещи, к которым пользователь может вернуться (остановленные activity), и, следовательно, более высокий приоритет имеет сохранение этих процессов.

Компоненты приложения

Activities

Activity обеспечивает следующие ключевые взаимодействия между системой и приложением:

- Помогает приложению обработать завершение процесса, чтобы пользователь мог вернуться к activity с восстановленным предыдущим состоянием.

- Предоставляет приложениям возможности реализовать пользовательские потоки между собой, а системе - координировать эти потоки. (Самый классический пример здесь – это кнопка share.)

Вы реализуете действие как подкласс класса Activity. Дополнительные сведения о классе Activity см. В руководстве разработчика Activity.

Компоненты приложения

Services

Service - это универсальная точка входа для поддержки работы приложения в фоновом режиме по разным причинам. Это компонент, который работает в фоновом режиме для выполнения длительных операций или выполнения работы для удаленных процессов.

Компоненты приложения

Services

Service не предоставляет пользовательский интерфейс. Например, service может воспроизводить музыку в фоновом режиме, когда пользователь находится в другом приложении, или может получать данные по сети, не блокируя взаимодействие пользователя с activity. Другой компонент, например activity, может запустить service и позволить ей работать или привязаться к ней для взаимодействия с ней.

Компоненты приложения

Services

На самом деле есть два очень разных по семантике service, которые сообщают системе, как управлять приложением:

- Запущенные service сообщают системе, что она должна поддерживать их работу до тех пор, пока их работа не будет завершена. Это может быть синхронизация некоторых данных в фоновом режиме или воспроизведение музыки даже после того, как пользователь покинет приложение.

Компоненты приложения

Services

Синхронизация данных в фоновом режиме или воспроизведение музыки также представляют два разных типа запущенных service, которые изменяют способ их обработки системой:

- Воспроизведение музыки - это то, о чем пользователь непосредственно осведомлен, поэтому приложение сообщает системе об этом, говоря, что оно хочет быть на переднем плане с уведомлением, чтобы сообщить об этом пользователю; в этом случае система знает, что ей следует очень постараться, чтобы процесс этого service продолжал работать, потому что пользователь будет недоволен, если он завершит работу.

Компоненты приложения

Services

Синхронизация данных в фоновом режиме или воспроизведение музыки также представляют два разных типа запущенных service, которые изменяют способ их обработки системой:

- Обычная фоновая служба не является чем-то, что пользователь непосредственно осознает как что- запущенное, поэтому система имеет больше свободы в управлении своим процессом. Это может позволить убить его (а затем перезапустить service через некоторое время), если ему нужна оперативная память для вещей, которые имеют более непосредственное отношение к пользователю.

Компоненты приложения

Services

Связанные service запускаются, потому что какое-то другое приложение (или система) заявило, что хочет использовать данный service. По сути, это service, предоставляющий API другому процессу. Таким образом, система знает, что между этими процессами существует зависимость, поэтому, если процесс A привязан к service в процессе B, она знает, что ему нужно поддерживать процесс B (и его service) работающим для A. Кроме того, если процесс A является чем-то, во что пользователь вовлечен, тогда он также знает, что процесс B должен быть чем-то, что также волнует пользователя.

Компоненты приложения

Services

Благодаря своей гибкости (к лучшему или худшему) service оказались действительно полезным строительным блоком для всех видов концепций систем более высокого уровня. Живые обои, прослушиватели уведомлений, хранители экрана, методы ввода, службы специальных возможностей и многие другие основные системные функции - все это построено как service, которые реализуются приложениями, и система привязывается к тому моменту, когда они должны быть запущены.

Компоненты приложения

Broadcast receivers

Broadcast receiver - это компонент, который позволяет системе доставлять события в приложение вне обычного пользовательского потока, позволяя приложению отвечать на общесистемные широковещательные объявления. Поскольку broadcast receiver являются еще одним четко определенным входом в приложение, система может доставлять широковещательные сообщения даже в приложения, которые в настоящее время не работают.

Компоненты приложения

Broadcast receivers

Так, например, приложение может запланировать будильник, чтобы опубликовать уведомление, чтобы сообщить пользователю о предстоящем событии ... и, доставив этот сигнал в broadcast receiver приложения, нет необходимости, чтобы приложение продолжало работать до тех пор, пока будильник срабатывает. Многие широковещательные сообщения исходят из системы - например, трансляция, извещающая о том, что экран выключился, разрядился аккумулятор или был сделан снимок. Приложения также могут инициировать широковещательные рассылки, например, чтобы сообщить другим приложениям, что некоторые данные были загружены на устройство и доступны для использования.

Компоненты приложения

Broadcast receivers

Хотя broadcast receiver не отображают пользовательский интерфейс, они могут создавать уведомление в строке состояния, чтобы предупредить пользователя о возникновении широковещательного события. Однако чаще всего broadcast receiver является просто шлюзом к другим компонентам и предназначен для выполнения минимального объема работы. Например, он может запланировать JobService для выполнения некоторой работы на основе события с JobScheduler.

Компоненты приложения

Content providers

Content provider управляет общим набором данных приложения, которые вы можете хранить в файловой системе, в базе данных SQLite, в Интернете или в любом другом постоянном хранилище, к которому ваше приложение может получить доступ. Через content provider другие приложения могут запрашивать или изменять данные, если content provider позволяет это.

Компоненты приложения

Content providers

Например, система Android предоставляет content provider, который управляет контактной информацией пользователя. Таким образом, любое приложение с соответствующими разрешениями может запрашивать content provider, например ContactsContract.Data, для чтения и записи информации о конкретном человеке.

Компоненты приложения

Content providers

Соблазнительно думать о content provider как об абстракции базы данных, потому что для этого распространенного случая в него встроено множество API и встроенная поддержка. Однако с точки зрения системного дизайна у них другое основное назначение. Для системы content provider - это точка входа в приложение для публикации именованных элементов данных, идентифицированных схемой URI. Таким образом, приложение может решить, как оно хочет сопоставить данные, которые оно содержит, с пространством имен URI, передавая эти URI другим объектам, которые, в свою очередь, могут использовать их для доступа к данным.

Компоненты приложения

Content providers

Это позволяет системе делать несколько вещей при управлении приложением:

- Назначение URI не требует, чтобы приложение продолжало работать, поэтому URI могут сохраняться после выхода из приложений, владеющих ими. Системе нужно только убедиться, что приложение-владелец все еще работает, когда ей нужно получить данные приложения из соответствующего URI.

Компоненты приложения

Content providers

Это позволяет системе делать несколько вещей при управлении приложением:

- Эти URI также обеспечивают важную детальную модель безопасности. Например, приложение может разместить URI для изображения в буфере обмена, но оставить content provider заблокированным, чтобы другие приложения не могли получить к нему свободный доступ. Когда второе приложение пытается получить доступ к этому URI в буфере обмена, система может разрешить этому приложению доступ к данным через временное разрешение URI, так что ему разрешен доступ к данным только за этим URI, но ни к чему другому во втором приложении.

Компоненты приложения

Content providers

Content provider также полезны для чтения и записи данных, которые являются личными для вашего приложения и не являются общими.

Компоненты приложения

Активация компонентов

Три из четырех типов компонентов - activity, service и broadcast reciever - активируются асинхронным сообщением, называемым intent. Intent связывают отдельные компоненты друг с другом во время выполнения. Вы можете думать о них как о мессенджерах, которые запрашивают действие от других компонентов, независимо от того, принадлежит ли компонент вашему приложению или другому.

Intent создается с помощью объекта Intent, который определяет сообщение для активации либо определенного компонента (явный intent), либо определенного типа компонента (неявный intent).

Компоненты приложения

Активация компонентов

Для activity и service intent определяет действие, которое необходимо выполнить (например, просмотреть или отправить что-либо), и может указывать URI данных, над которыми нужно действовать, среди прочего, что запускаемому компоненту может потребоваться знать. Например, intent может передавать запрос на activity, чтобы показать изображение или открыть веб-страницу. В некоторых случаях вы можете запустить activity, чтобы получить результат, и в этом случае activity также возвращает результат в intent. Например, вы можете объявить intent позволить пользователю выбрать личный контакт и вернуть его вам. Возвращаемый intent включает URI, указывающий на выбранный контакт.

Компоненты приложения

Активация компонентов

Для broadcast receiver intent просто определяет транслируемое объявление. Например, широковещательное сообщение, указывающее, что батарея устройства разряжена, включает в себя только известную строку действия, которая указывает, что батарея разряжена.

Компоненты приложения

Активация компонентов

В отличие от activity, service и broadcast receiver, content provider не активируются intent. Скорее, они активируются по запросу content resolver. Content resolver обрабатывает все прямые транзакции с content provider так, что компонент, выполняющий транзакции с content provider, не требуется, и вместо этого он вызывает методы объекта content resolver. Это оставляет уровень абстракции между content provider и компонентом, запрашивающим информацию (для безопасности).

Компоненты приложения

Активация компонентов

Существуют отдельные методы активации каждого типа компонентов:

- Вы можете запустить activity или дать ему что-то новое, передав intent в startActivity () или startActivityForResult () (если вы хотите, чтобы действие возвращало результат).

- В Android 5.0 (уровень API 21) и более поздних версиях вы можете использовать класс JobScheduler для планирования действий. Для более ранних версий Android вы можете запустить service (или дать новые инструкции действующему service), передав intent в startService (). Вы можете выполнить привязку к service, передав intent в bindService ().

Компоненты приложения

Активация компонентов

Существуют отдельные методы активации каждого типа компонентов:

- Вы можете инициировать широковещательную рассылку, передав intent таким методам, как sendBroadcast (), sendOrderedBroadcast () или sendStickyBroadcast ().

- Вы можете выполнить запрос к content provider, вызвав query () для ContentResolver.

Компоненты приложения

Файл манифеста

Прежде чем система Android сможет запустить компонент приложения, система должна знать, что компонент существует, прочитав файл манифеста приложения AndroidManifest.xml. Ваше приложение должно объявить все свои компоненты в этом файле, который должен находиться в корне каталога проекта приложения.

Компоненты приложения

Файл манифеста

Манифест выполняет ряд функций в дополнение к объявлению компонентов приложения, например:

- Определяет любые разрешения пользователя, которые требуются приложению, такие как доступ в Интернет или доступ для чтения к контактам пользователя.
- Объявляет минимальный уровень API, необходимый для приложения, в зависимости от того, какие API использует приложение.
- Объявляет аппаратные и программные функции, используемые или требуемые приложением, такие как камера, службы Bluetooth или сенсорный экран.
- Объявляет библиотеки API, с которыми необходимо связать приложение (кроме API-интерфейсов Android framework), например библиотеку Google Maps.

Компоненты приложения

Объявление компонентов

Основная задача манифеста - информировать систему о компонентах приложения. Например, файл манифеста может объявлять действие следующим образом:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ... >
    <application android:icon="@drawable/app_icon.png" ... >
        <activity android:name="com.example.project.ExampleActivity"
                  android:label="@string/example_label" ... >
        </activity>
        ...
    </application>
</manifest>
```

Компоненты приложения

Объявление компонентов

В элементе <application> атрибут android: icon указывает на ресурсы для значка, который идентифицирует приложение.

В элементе <activity> атрибут android: name указывает полное имя класса подкласса Activity, а атрибут android: label указывает строку, которая будет использоваться в качестве видимой для пользователя метки для действия.

Компоненты приложения

Объявление компонентов

Вы должны объявить все компоненты приложения, используя следующие элементы:

- <activity> элементы для activity.
- <service> элементы для service.
- <receiver> элементы для broadcast receiver.
- <provider> элементы для content provider.

Компоненты приложения

Объявление компонентов

Activity, service и content provider, которые вы включаете в свой исходный код, но не объявляете в манифесте, не видны системе и, следовательно, никогда не могут быть запущены. Однако broadcast receiver могут быть либо объявлены в манифесте, либо созданы динамически в коде как объекты BroadcastReceiver и зарегистрированы в системе путем вызова registerReceiver ().

Компоненты приложения

Объявление возможностей компонентов

Как говорилось выше, в разделе «Активация компонентов» вы можете использовать intent для запуска activity, service и broadcast receiver. Вы можете использовать intent, явно назвав целевой компонент (используя имя класса компонента) в intent. Вы также можете использовать неявный intent, который описывает тип действия, которое нужно выполнить, и, при необходимости, данные, над которыми вы хотите выполнить действие. Неявный intent позволяет системе найти на устройстве компонент, который может выполнить действие, и запустить его. Если есть несколько компонентов, которые могут выполнять действие, описанное в intent, пользователь выбирает, какой из них использовать.

Компоненты приложения

Объявление возможностей компонентов

Система определяет компоненты, которые могут реагировать на intent, сравнивая полученный intent с intent filters, предоставленными в файле манифеста других приложений на устройстве.

Когда вы объявляете действие в манифесте приложения, вы можете дополнительно включить intent filters, которые объявляют возможности действия, чтобы оно могло реагировать на intent других приложений. Вы можете объявить intent filters для своего компонента, добавив элемент <intent-filter> в качестве дочернего элемента объявления компонента.

Компоненты приложения

Объявление возможностей компонентов

Например, если вы создаете приложение электронной почты с действием для создания нового электронного письма, вы можете объявить intent filter для ответа на intent «отправить» (чтобы отправить новое электронное письмо), как показано в следующем примере:

```
<manifest ... >
    ...
    <application ... >
        <activity android:name="com.example.project.ComposeEmailActivity">
            <intent-filter>
                <action android:name="android.intent.action.SEND" />
                <data android:type="*/*" />
                <category android:name="android.intent.category.DEFAULT" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

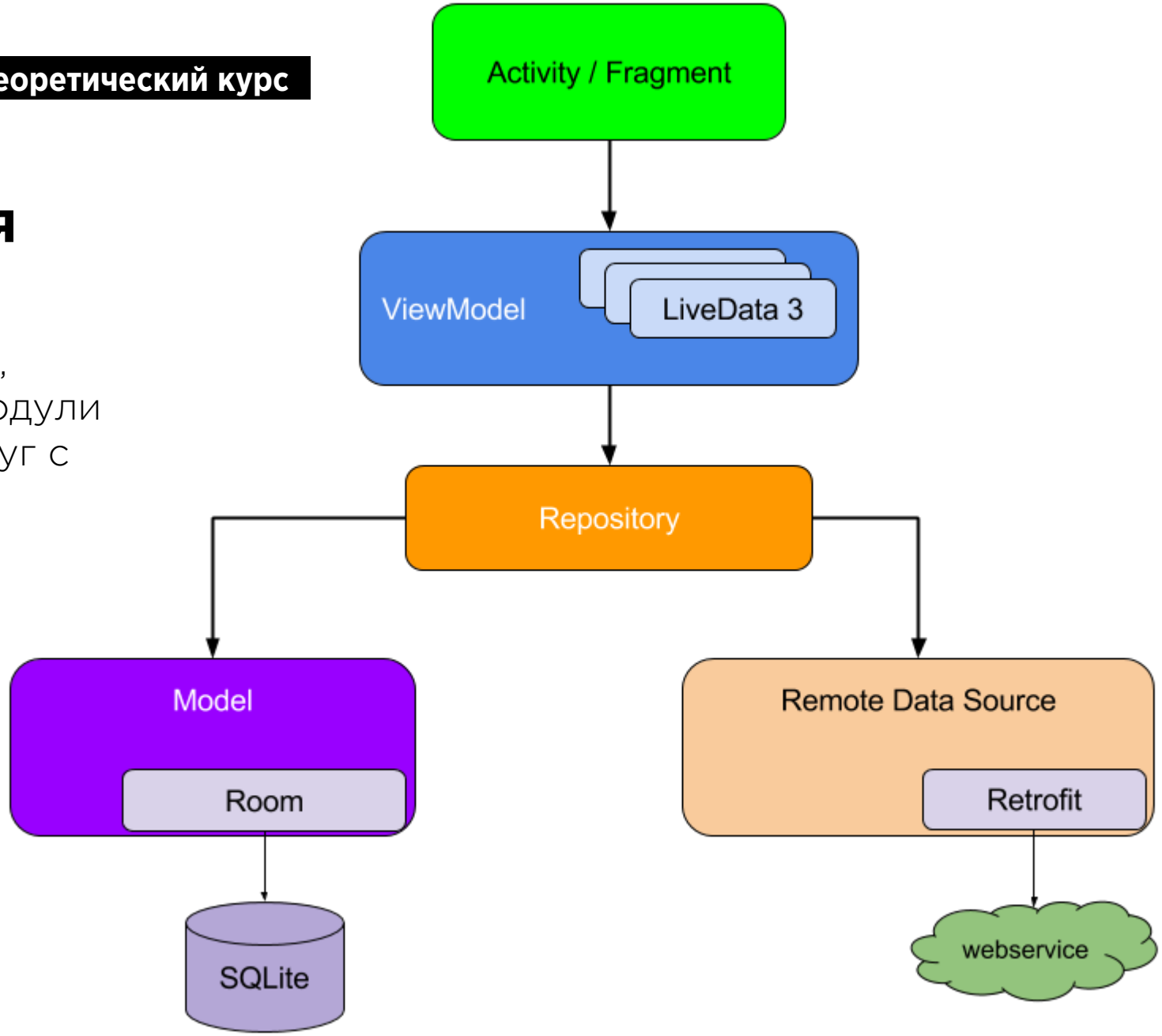
Компоненты приложения

Объявление возможностей компонентов

Если другое приложение создает intent с действием ACTION_SEND и передает его startActivity (), система может запустить ваше activity, чтобы пользователь мог составить черновик и отправить электронное письмо.

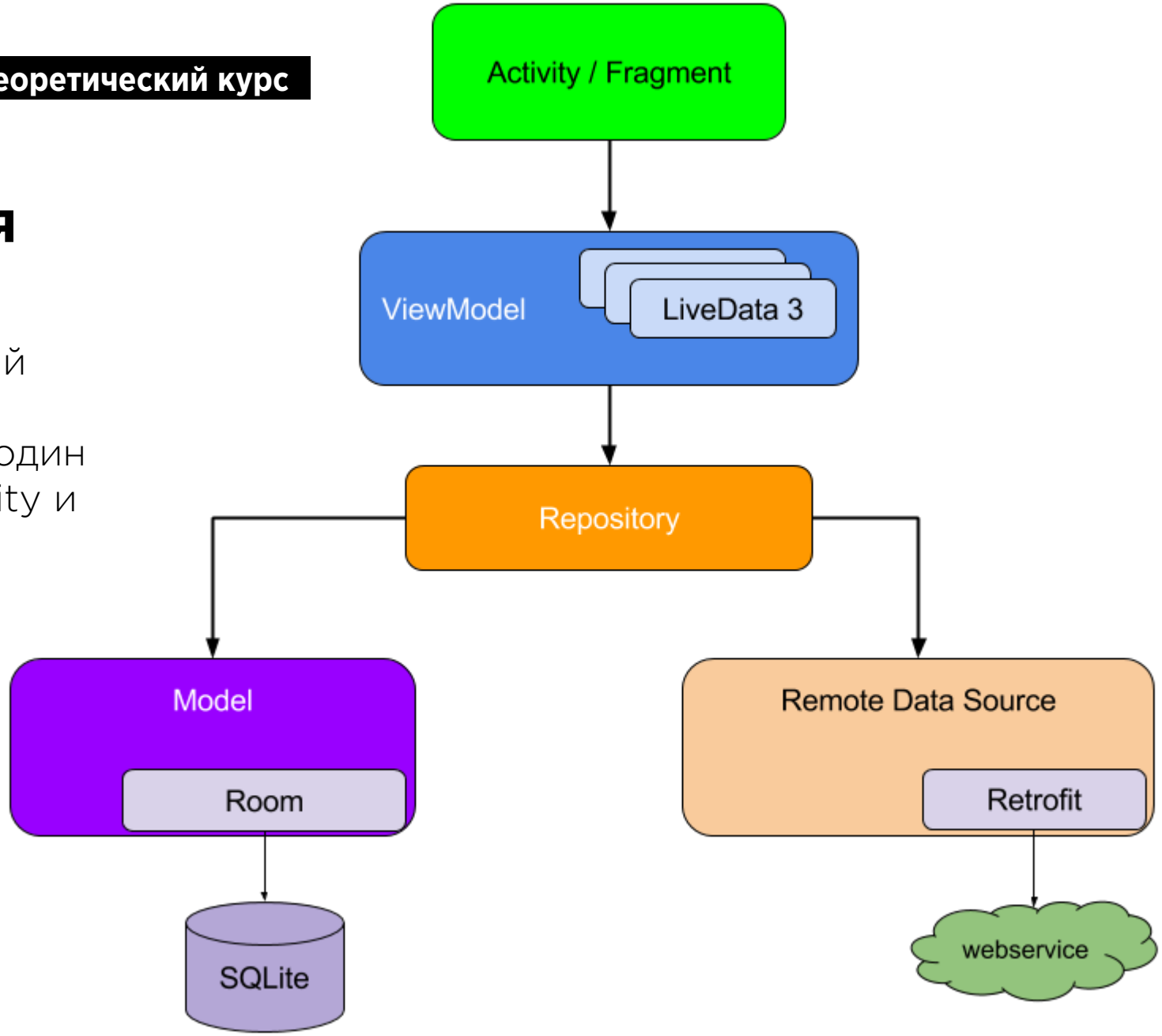
Компоненты приложения

Рассмотрим следующую схему, которая показывает, как все модули должны взаимодействовать друг с другом после разработки приложения:



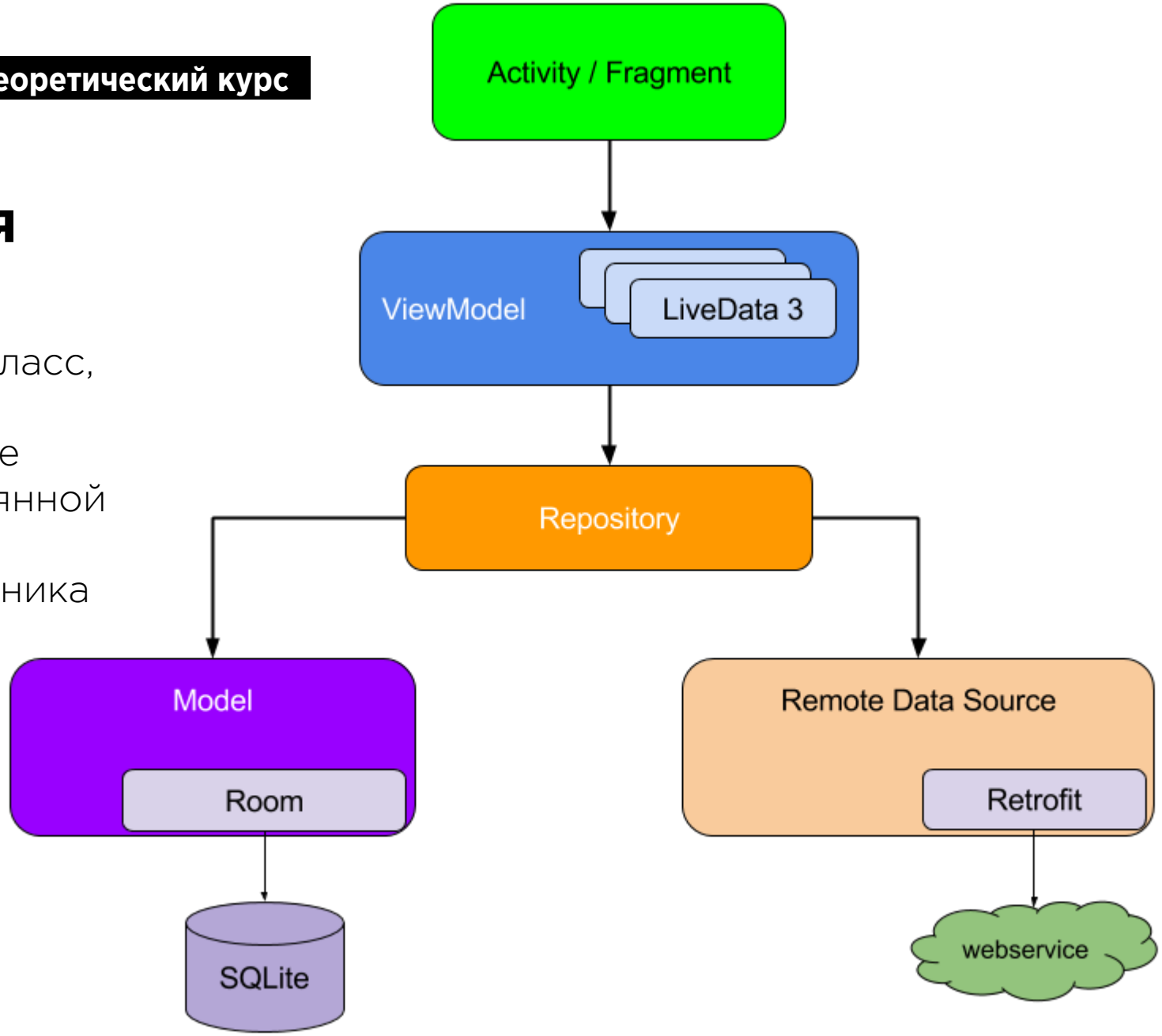
Компоненты приложения

Обратите внимание, что каждый компонент зависит только от компонента, находящегося на один уровень ниже. Например, activity и fragment зависят только от ViewModel.



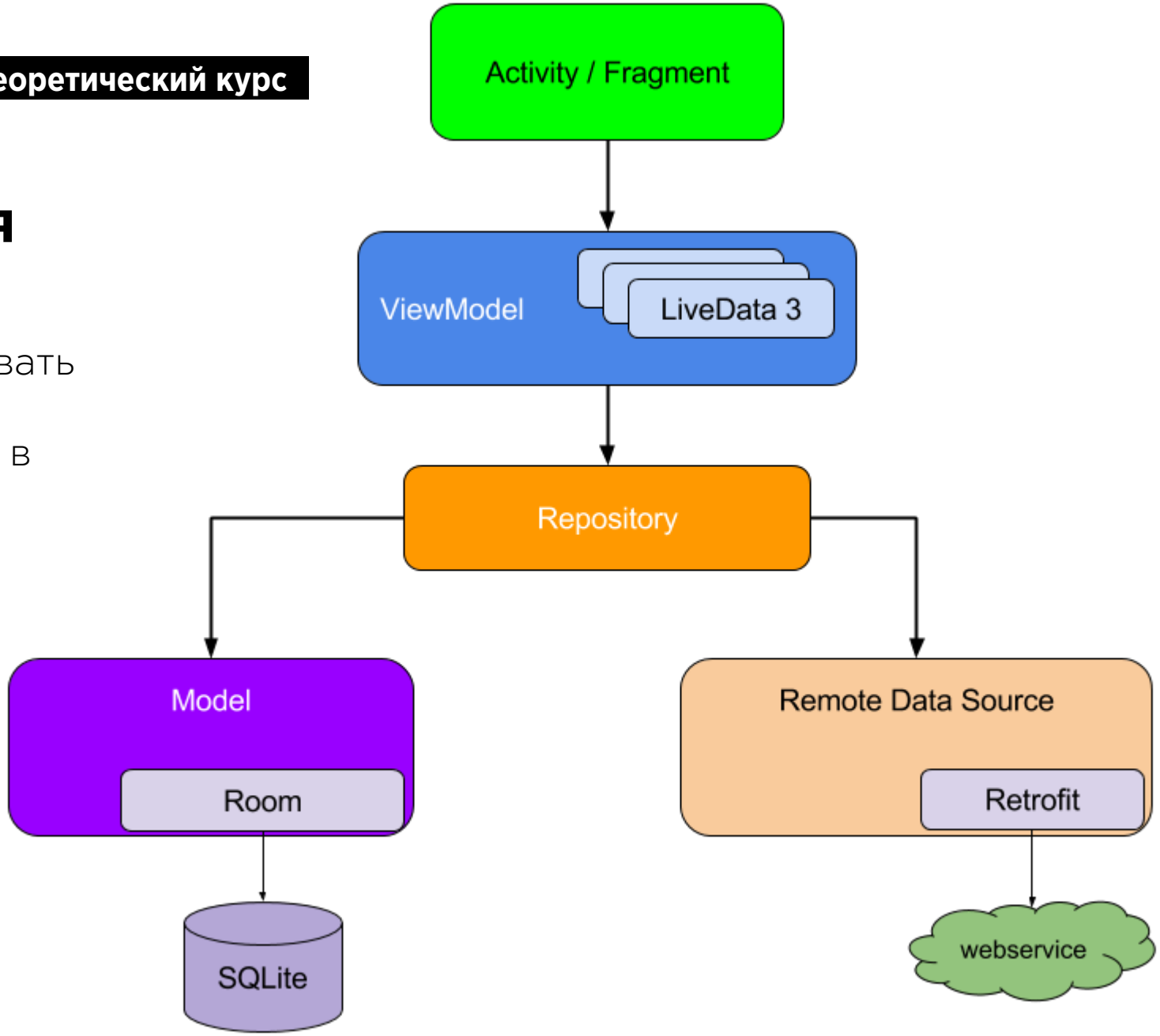
Компоненты приложения

Репозиторий - единственный класс, который зависит от множества других классов; в этом примере репозиторий зависит от постоянной модели данных (Model) и удаленного внутреннего источника данных (RDS).



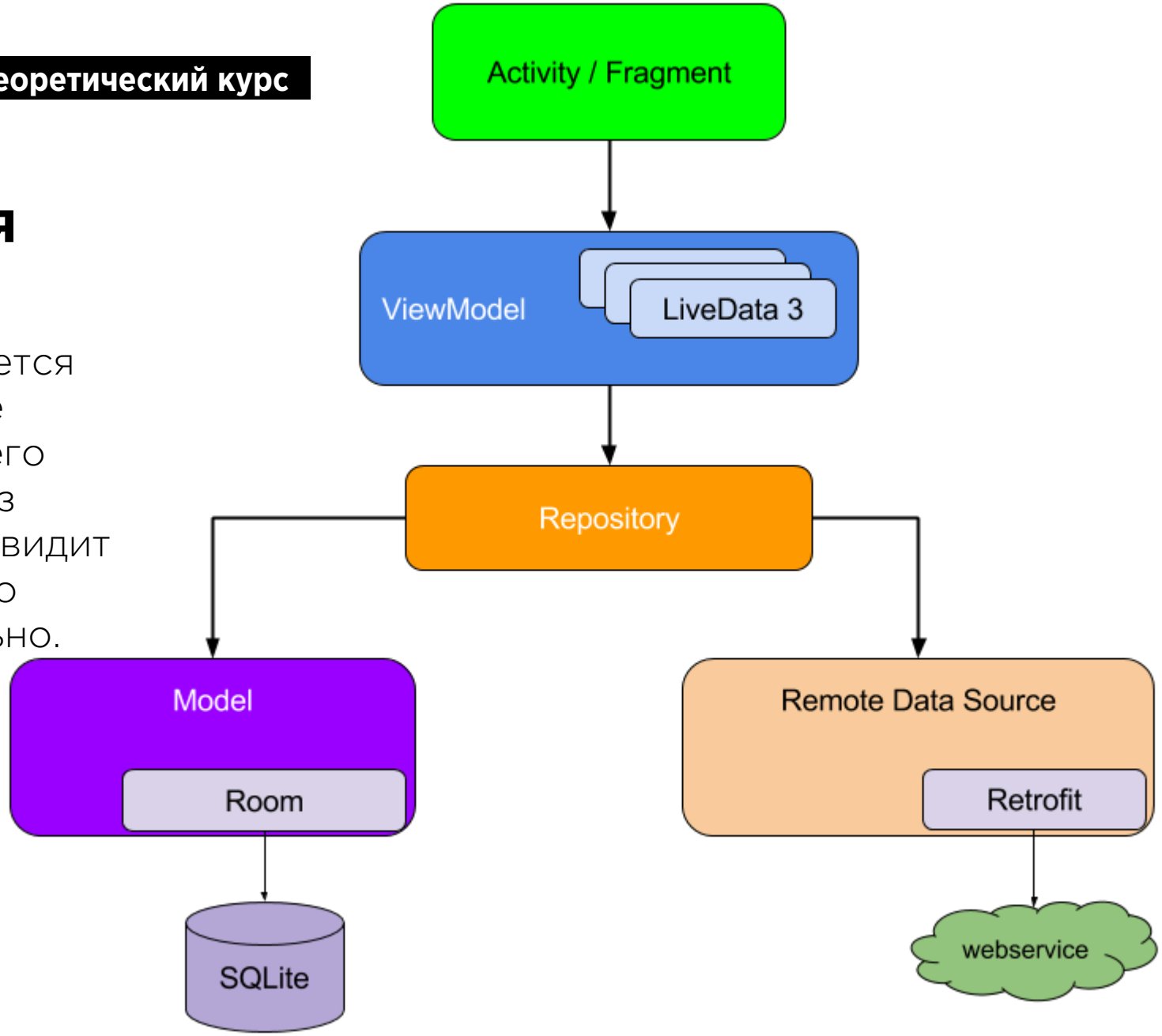
Компоненты приложения

Такой подход позволяет создавать последовательные и приятные пользовательские интерфейсы в приложении.



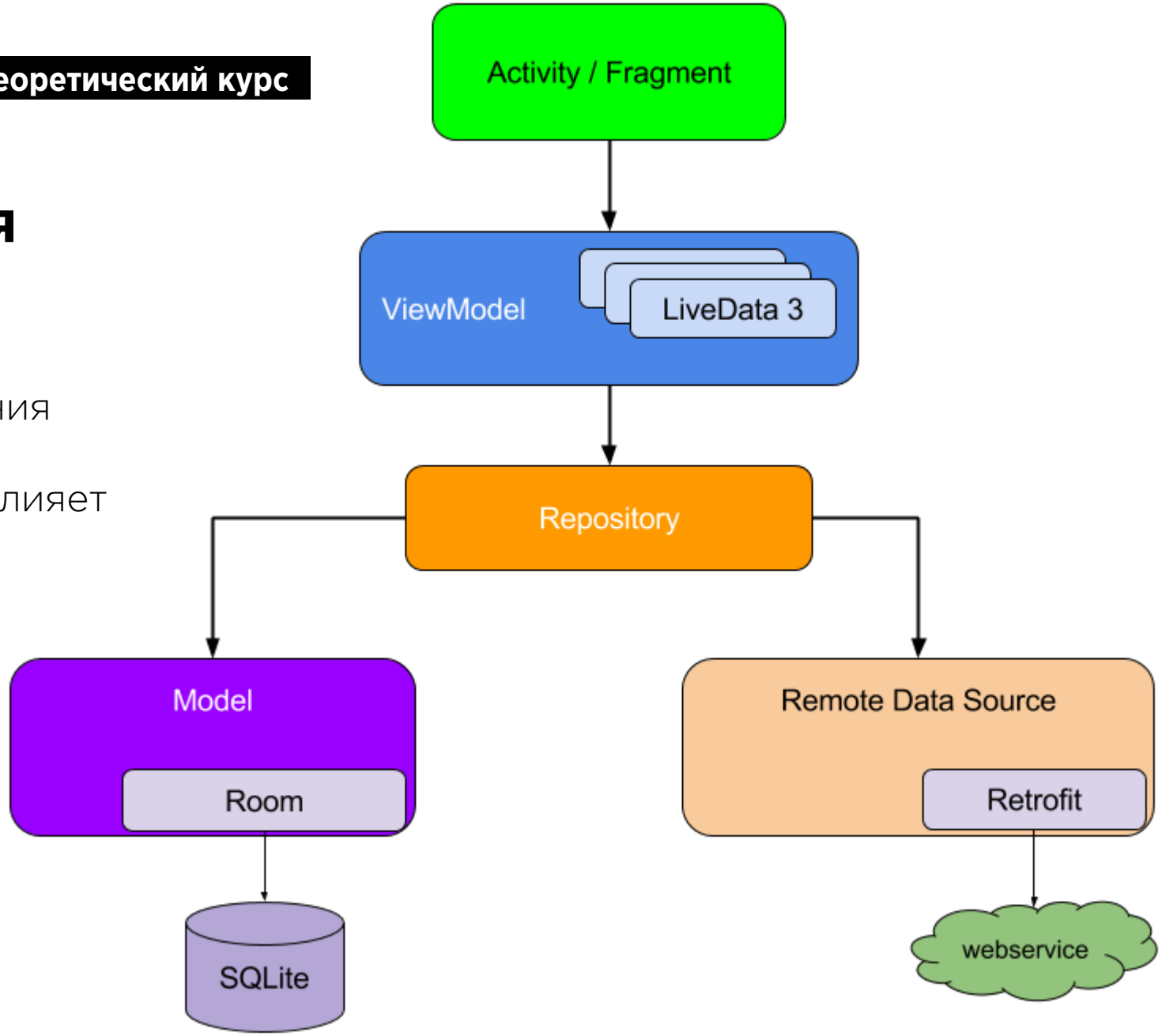
Компоненты приложения

Независимо от того, возвращается ли пользователь в приложение через несколько минут после его последнего закрытия или через несколько дней, он мгновенно видит информацию, которую для него приложение сохранило локально.



Компоненты приложения

Если же эти данные устарели, модуль репозитория приложения начинает обновлять данные в фоновом режиме. И это не повлияет на опыт пользователя при использовании приложения.



Жизненный цикл Activity

Activity - это единичная сфокусированная вещь, с которой может взаимодействовать пользователь. Почти все activity взаимодействуют с пользователем, поэтому класс Activity позаботится о создании для пользователя окна, в котором вы можете разместить свой пользовательский интерфейс с помощью setContentView (View). Хотя activity часто представляются пользователю как полноэкранные окна, их также можно использовать и другими способами: как плавающие окна (через тему с установленным R.attr.windowIsFloating), многооконный режим или встроенные в другие окна.

Жизненный цикл Activity

Почти все подклассы Activity реализуют два метода:

- onCreate (Bundle) - это место, где вы инициализируете свою activity. Что наиболее важно, здесь вы обычно вызываете setContentView (int) с ресурсом макета, определяющим ваш пользовательский интерфейс, и используете findViewById (int) для извлечения виджетов в этом пользовательском интерфейсе, с которыми вам нужно взаимодействовать программно.

- onPause () - это то место, где вы имеете дело с пользователем, который приостанавливает активное взаимодействие с activity. На этом этапе любые изменения, сделанные пользователем, должны быть зафиксированы (обычно в ContentProvider, хранящем данные). В этом состоянии activity все еще отображается на экране.

Жизненный цикл Activity

Для использования с Context.startActivity () все классы activity должны иметь соответствующее объявление <activity> в своем пакете AndroidManifest.xml.

Жизненный цикл Activity

Activity в системе управляются как стеки операций. Когда новое activity запускается, оно обычно помещается в верхнюю часть текущего стека и становится выполняемым activity - предыдущее activity всегда остается под ним в стеке и не выходит на передний план снова, пока новое activity не завершится. На экране может быть один или несколько стеков activity.

Про стек операций поговорим отдельно в дальнейшем.

Жизненный цикл Activity

По сути, activity имеет четыре состояния:

- Если activity находится на переднем плане экрана (в наивысшей позиции самого верхнего стека), оно активно или выполняется. Обычно это activity, с которым в данный момент взаимодействует пользователь.

- Если activity потеряло фокус, но все еще отображается пользователю, оно отображается. Это возможно, если новое не полноразмерное или прозрачное activity фокусируется на вершине вашего activity, другое activityзанимает более высокое положение в многооконном режиме или само activity не может быть сфокусировано в текущем оконном режиме. Такая activity полностью активна (она поддерживает всю информацию о состоянии и членах и остается прикрепленной к оконному менеджеру).

Жизненный цикл Activity

По сути, activity имеет четыре состояния:

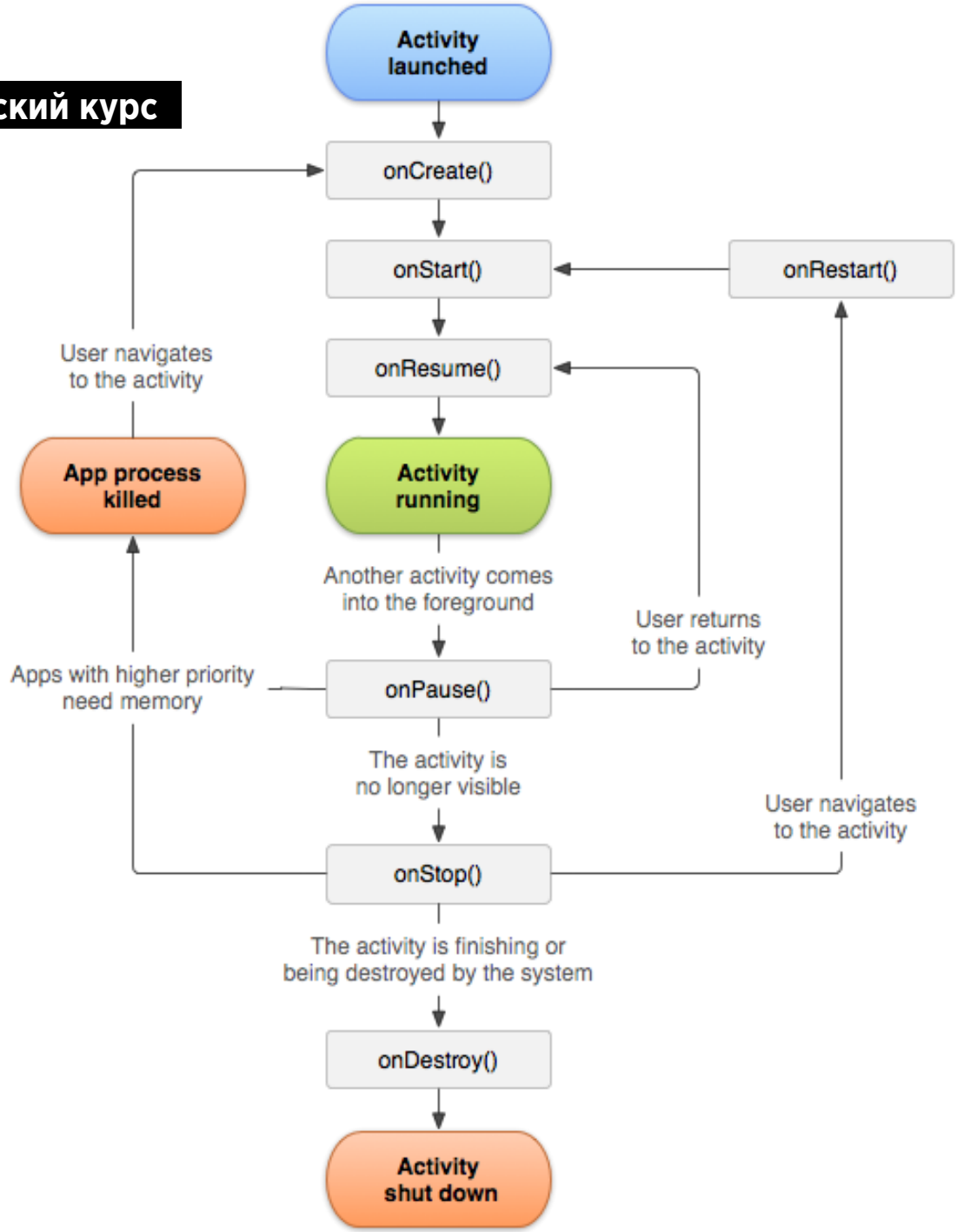
- Если activity полностью скрыто другим activity, оно останавливается или скрывается. Он по-прежнему сохраняет всю информацию о состоянии и членах, однако больше не видена пользователю, поэтому её окно скрыто, и она скорее всего будет уничтожена системой, когда память понадобится в другом месте.

- Система может удалить activity из памяти, либо попросив её завершиться, либо просто прервав процесс, сделав убив его. Когда она снова отображается для пользователя, она должна быть полностью перезапущена и восстановлена до предыдущего состояния.

Жизненный цикл Activity

На следующей диаграмме показаны важные пути состояния Activity. Квадратные прямоугольники представляют методы обратного вызова (callbacks), которые вы можете реализовать для выполнения операций, когда Activity перемещается между состояниями. Цветные овалы - это основные состояния, в которых может находиться Activity.

Жизненный цикл Activity



Жизненный цикл Activity

Есть три ключевых цикла, в мониторинге которых вы можете быть заинтересованы:

- Все время существования activity происходит между первым вызовом onCreate (Bundle) и одним финальным вызовом onDestroy (). Activity выполнит настройку «глобального» состояния в onCreate () и освободит все оставшиеся ресурсы в onDestroy (). Например, если у него есть поток, работающий в фоновом режиме для загрузки данных из сети, он может создать этот поток в onCreate (), а затем остановить поток в onDestroy ().

Жизненный цикл Activity

Есть три ключевых цикла, в мониторинге которых вы можете быть заинтересованы:

- Видимое время жизни activity происходит между вызовом onStart () до соответствующего вызова onStop (). В это время пользователь может видеть activity на экране, хотя она может не находиться на переднем плане и взаимодействовать с пользователем. Между этими двумя методами вы можете поддерживать ресурсы, необходимые для демонстрации activity пользователю. Например, вы можете зарегистрировать BroadcastReceiver в onStart () для отслеживания изменений, влияющих на ваш пользовательский интерфейс, и отменить регистрацию в onStop (), когда пользователь больше не видит то, что вы показываете. Методы onStart () и onStop () можно вызывать несколько раз, поскольку activity становится видимым и скрытым для пользователя.

Жизненный цикл Activity

Есть три ключевых цикла, в мониторинге которых вы можете быть заинтересованы:

- Время жизни переднего плана activity происходит между вызовом onResume () до соответствующего вызова onPause (). В это время activity видна, активна и взаимодействует с пользователем. Activity часто может переходить между состояниями возобновления и приостановки - например, когда устройство переходит в спящий режим, когда доставляется результат activity, когда доставляется новый intent, - поэтому код в этих методах должен быть довольно легким.

Жизненный цикл Activity

Весь жизненный цикл activity определяется следующими методами Activity. Все это перехватчики, которые вы можете переопределить для выполнения соответствующей работы при изменении состояния activity. Все activity реализуют onCreate (Bundle) для первоначальной настройки; многие также реализуют onPause () для фиксации изменений данных и подготовки к приостановке взаимодействия с пользователем, и onStop () для обработки, которая больше не отображается на экране. Вы всегда должны вызывать свой суперкласс при реализации этих методов.

Жизненный цикл Activity

Метод	Описание	Killable?	Следующий
onCreate()	Вызывается при первом создании activity. Здесь вы должны выполнять все ваши обычные статические настройки: создавать представления, связывать данные со списками и т. д. Этот метод также предоставляет вам Bundle, содержащий ранее замороженное состояние activity, если оно было.	Нет	onStart()
onRestart()	Вызывается после того, как ваша activity была остановлена, до ее повторного запуска.	Нет	onStart()
onStart()	Вызывается, когда activity становится видимым для пользователя. За ним следует onResume (), если действие выходит на передний план, или onStop (), если оно становится скрытым.	Нет	onResume() или onStop()

Жизненный цикл Activity

Метод	Описание	Killable?	Следующий
onResume()	Вызывается, когда activity начинает взаимодействовать с пользователем. На этом этапе ваша activity находится на вершине своего стека действий, и в нее поступает пользовательский ввод.	Нет	onPause()
onPause()	Вызывается, когда activity теряет состояние переднего плана, становится недоступным для фокусировки или перед переходом в состояние остановлено / скрыто или уничтожено. Activity по-прежнему видны пользователю, поэтому рекомендуется поддерживать её визуально активной и продолжать обновлять пользовательский интерфейс. Реализации этого метода должны быть очень быстрыми, потому что следующее activity не будет возобновлено, пока этот метод не вернется. За ним следует либо onResume (), если activity возвращается на передний план, либо onStop (), если оно становится невидимым для пользователя.	Зависит от версии API	onResume() или onStop()

Жизненный цикл Activity

Метод	Описание	Killable?	Следующий
onStop()	Вызывается, когда activity больше не отображается для пользователя. Это может произойти либо из-за того, что новая activity запускается поверх, либо из-за того, что существующее перемещается вперед нее, либо она уничтожается. Обычно это используется для остановки анимации, обновления пользовательского интерфейса и т. д. Затем следует либо onRestart (), если эта activity возвращается для взаимодействия с пользователем, либо onDestroy (), если эта activity прекращается.	Да	onRestart() или onDestroy()
onDestroy()	Последний вызов, который вы получите перед тем, как ваша activity будет уничтожена. Это может произойти либо из-за того, что activity завершается (кто-то вызвал Activity # finish), либо из-за того, что система временно уничтожает этот экземпляр activity для экономии места. Вы можете различать эти два сценария с помощью метода isFinishing ().	Да	-

Жизненный цикл Activity

Обратите внимание на столбец «Killable» в приведенной выше таблице - для тех методов, которые отмечены как подлежащие уничтожению, после того, как этот метод вернет процесс, в котором выполняется activity, система может в любое время убить ее без выполнения другой строки его кода. Из-за этого вы должны использовать метод onPause () для записи любых постоянных данных (например, изменений, внесенных пользователем) в хранилище.

Кроме того, метод onSaveInstanceState (android.os.Bundle) вызывается перед помещением activity в такое фоновое состояние, что позволяет вам сохранить любое динамическое состояние экземпляра вашей activity в Bundle, чтобы впоследствии получить его в onCreate (Bundle), если activity необходимо воссоздать.

Жизненный цикл Activity

Обратите внимание, что важно сохранять постоянные данные в onPause () вместо onSaveInstanceState (Bundle), потому что последний не является частью обратных вызовов жизненного цикла, поэтому не будет вызываться в каждой ситуации, как описано в документации.

Жизненный цикл Activity

Для тех методов, которые не помечены как подлежащие уничтожению, процесс activity не будет уничтожен системой, начиная с момента вызова метода и продолжаясь после его возврата. Таким образом, activity находится в состоянии уничтожения, например, между периодом после onStop () и до начала onResume (). Имейте в виду, что при чрезмерной нехватке памяти система может в любой момент остановить процесс приложения.

Жизненный цикл Fragment

У каждого экземпляра фрагмента свой жизненный цикл. Когда пользователь перемещается и взаимодействует с вашим приложением, ваши фрагменты переходят через различные состояния в своем жизненном цикле, когда они добавляются, удаляются, а также выходят на экран или выходят из него.

Для управления жизненным циклом Fragment реализует LifecycleOwner, предоставляя объект Lifecycle, к которому вы можете получить доступ с помощью метода getLifecycle ().

Жизненный цикл Fragment

Каждое возможное состояние жизненного цикла представлено в перечислении Lifecycle.State.

- INITIALIZED
- CREATED
- STARTED
- RESUMED
- DESTROYED

Жизненный цикл Fragment

Создав фрагмент поверх Lifecycle, вы можете использовать методы и классы, доступные с помощью Handling Lifecycles with Lifecycle-Aware Components. Например, вы можете отобразить местоположение устройства на экране с помощью компонента, учитывающего жизненный цикл. Этот компонент может автоматически начинать прослушивание, когда фрагмент становится активным, и останавливаться, когда фрагмент переходит в неактивное состояние.

В качестве альтернативы использованию LifecycleObserver класс Fragment включает методы обратного вызова, которые соответствуют каждому из изменений в жизненном цикле фрагмента. К ним относятся onCreate (), onStart (), onResume (), onPause (), onStop () и onDestroy ().

Жизненный цикл Fragment

Представления (View) в фрагменте имеет отдельный жизненный цикл, который управляется независимо от жизненного цикла фрагмента. Фрагменты поддерживают LifecycleOwner для своего представления, доступ к которому можно получить с помощью getViewLifecycleOwner () или getViewLifecycleOwnerLiveData (). Доступ к жизненному циклу представления полезен в ситуациях, когда компонент, учитывающий жизненный цикл, должен выполнять работу, только пока существует представление фрагмента, например, для наблюдения за LiveData, которая предназначена только для отображения на экране.

Далее подробно рассмотрим жизненный цикл фрагмента, некоторые правила, определяющие состояние жизненного цикла фрагмента, и посмотрим взаимосвязь между состояниями жизненного цикла и обратными вызовами жизненного цикла фрагмента

Жизненный цикл Fragment

Когда создается экземпляр фрагмента, он запускается в состоянии INITIALIZED. Чтобы фрагмент прошел оставшуюся часть своего жизненного цикла, его необходимо добавить в FragmentManager. FragmentManager отвечает за определение состояния, в котором должен находиться его фрагмент, а также за его перемещение в это состояние.

Помимо жизненного цикла фрагмента, FragmentManager также отвечает за присоединение фрагментов к host-activity и их отсоединение, когда фрагмент больше не используется. Класс Fragment имеет два метода обратного вызова, onAttach () и onDetach (), которые можно переопределить для выполнения работы при возникновении любого из этих событий.

Жизненный цикл Fragment

Обратный вызов onAttach () вызывается, когда фрагмент добавляется в FragmentManager и присоединяется к его host-activity. На данный момент фрагмент активен, и FragmentManager управляет его состоянием жизненного цикла. На этом этапе методы FragmentManager, такие как findFragmentById (), возвращают этот фрагмент.

onAttach () всегда вызывается перед изменением состояния жизненного цикла.

Жизненный цикл Fragment

Обратный вызов onDetach () вызывается, когда фрагмент был удален из FragmentManager и отсоединен от его host-activity. Фрагмент больше не активен и больше не может быть получен с помощью findFragmentById ().

onDetach () всегда вызывается после любого изменения состояния жизненного цикла.

Обратите внимание, что эти обратные вызовы не связаны с методами Attach () и Detach () FragmentTransaction.

Жизненный цикл Fragment

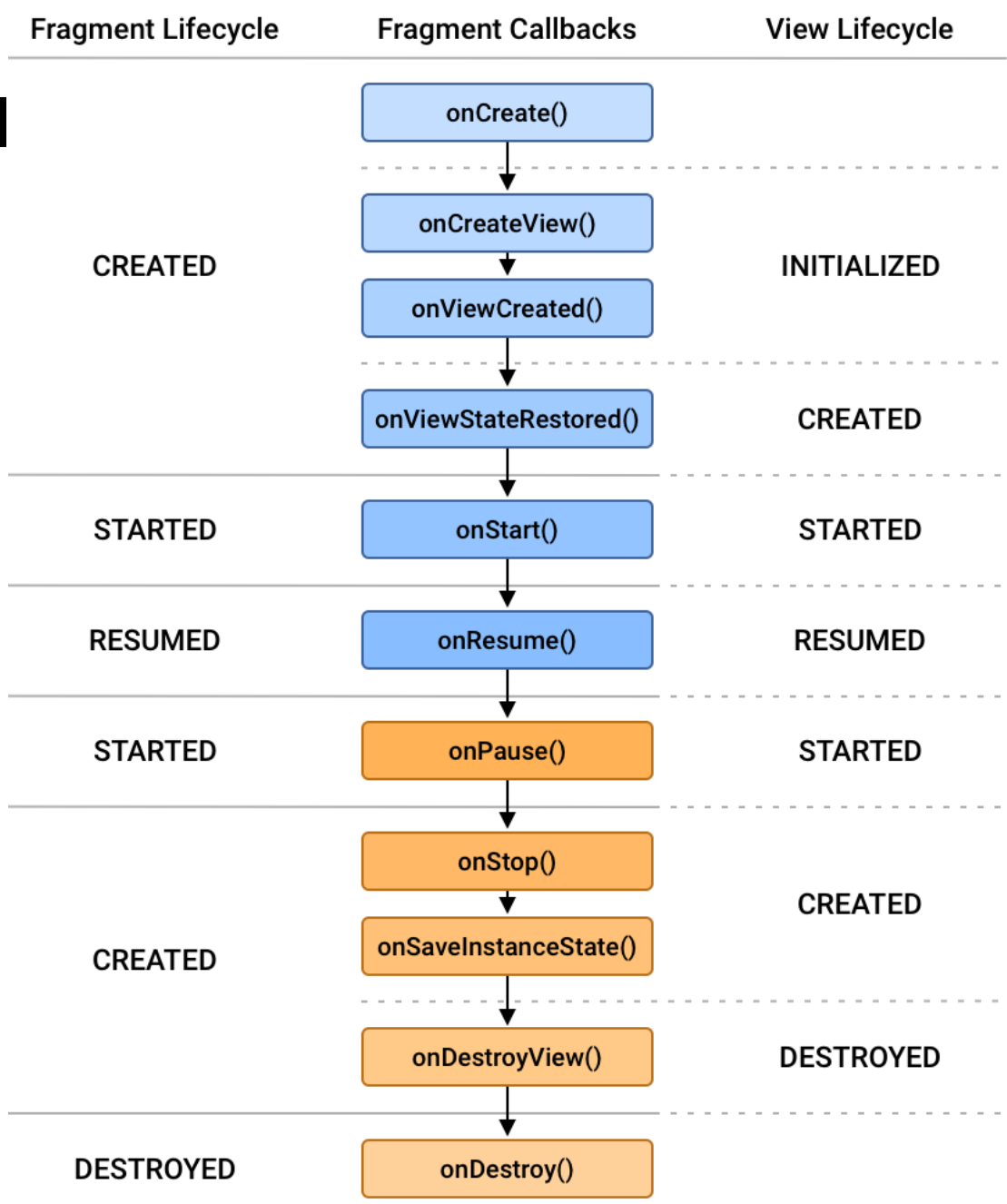
При определении состояния жизненного цикла фрагмента FragmentManager учитывает следующее:

- Максимальное состояние фрагмента определяется его FragmentManager. Фрагмент не может развиваться дальше состояния своего FragmentManager.
- В рамках FragmentTransaction вы можете установить максимальное состояние жизненного цикла для фрагмента с помощью setMaxLifecycle ().
- Состояние жизненного цикла фрагмента никогда не может быть выше, чем у его родителя. Например, родительский фрагмент или activity должны быть запущены до его дочерних фрагментов. Точно так же дочерние фрагменты должны быть остановлены до их родительского фрагмента или activity.

Жизненный цикл Fragment

На рисунке показано каждое из состояний жизненного цикла фрагмента и их отношение как к обратным вызовам жизненного цикла фрагмента, так и к жизненному циклу представления фрагмента.

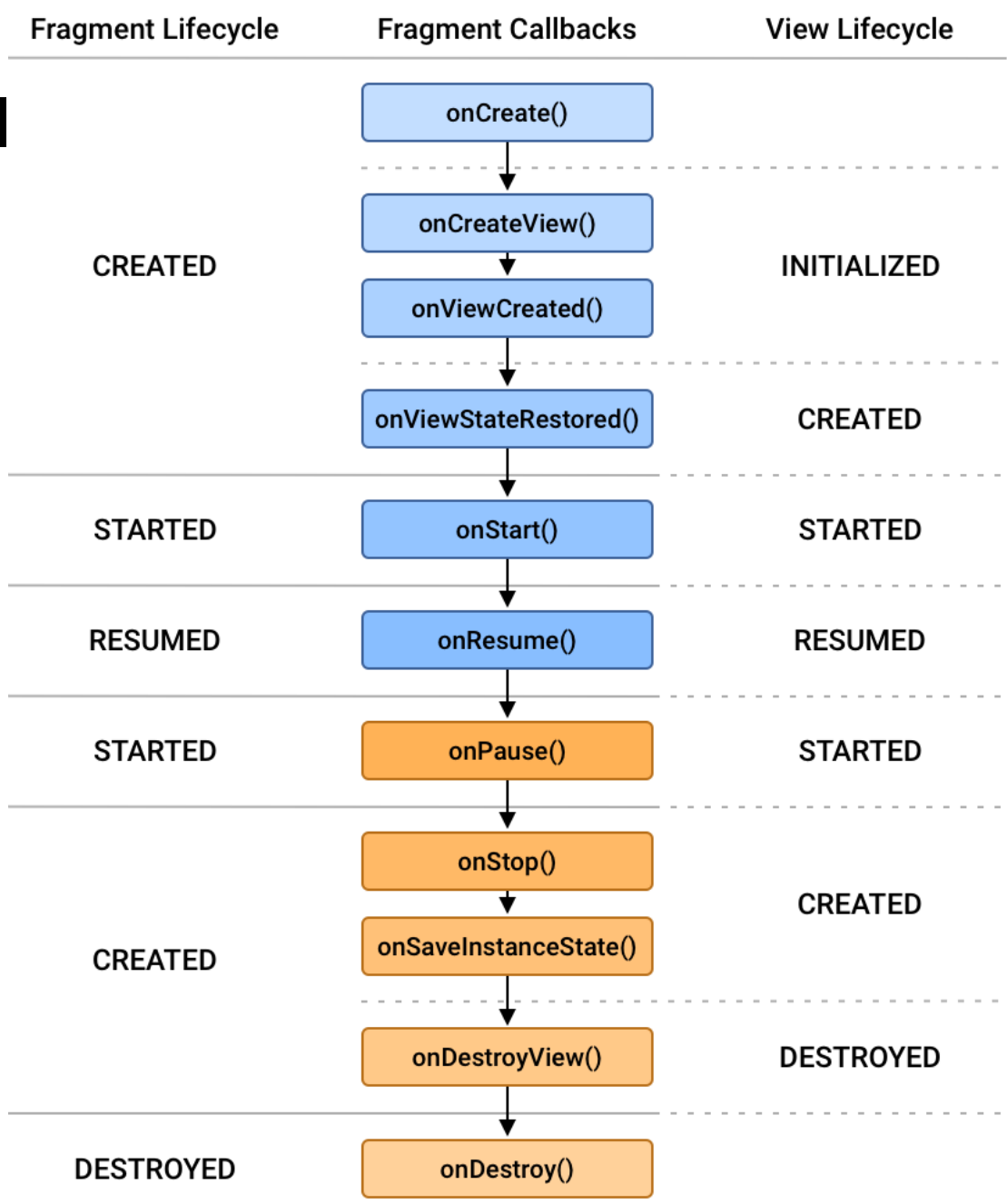
.



Жизненный цикл Fragment

По мере того, как фрагмент проходит свой жизненный цикл, он перемещается вверх и вниз по своим состояниям.

Например, фрагмент, добавленный в верхнюю часть backstack, перемещается вверх от CREATED к STARTED к RESUMED. И наоборот, когда фрагмент извлекается из backstack, он перемещается вниз по этим состояниям, переходя от RESUMED к STARTED, CREATED и, наконец, DESTROYED.



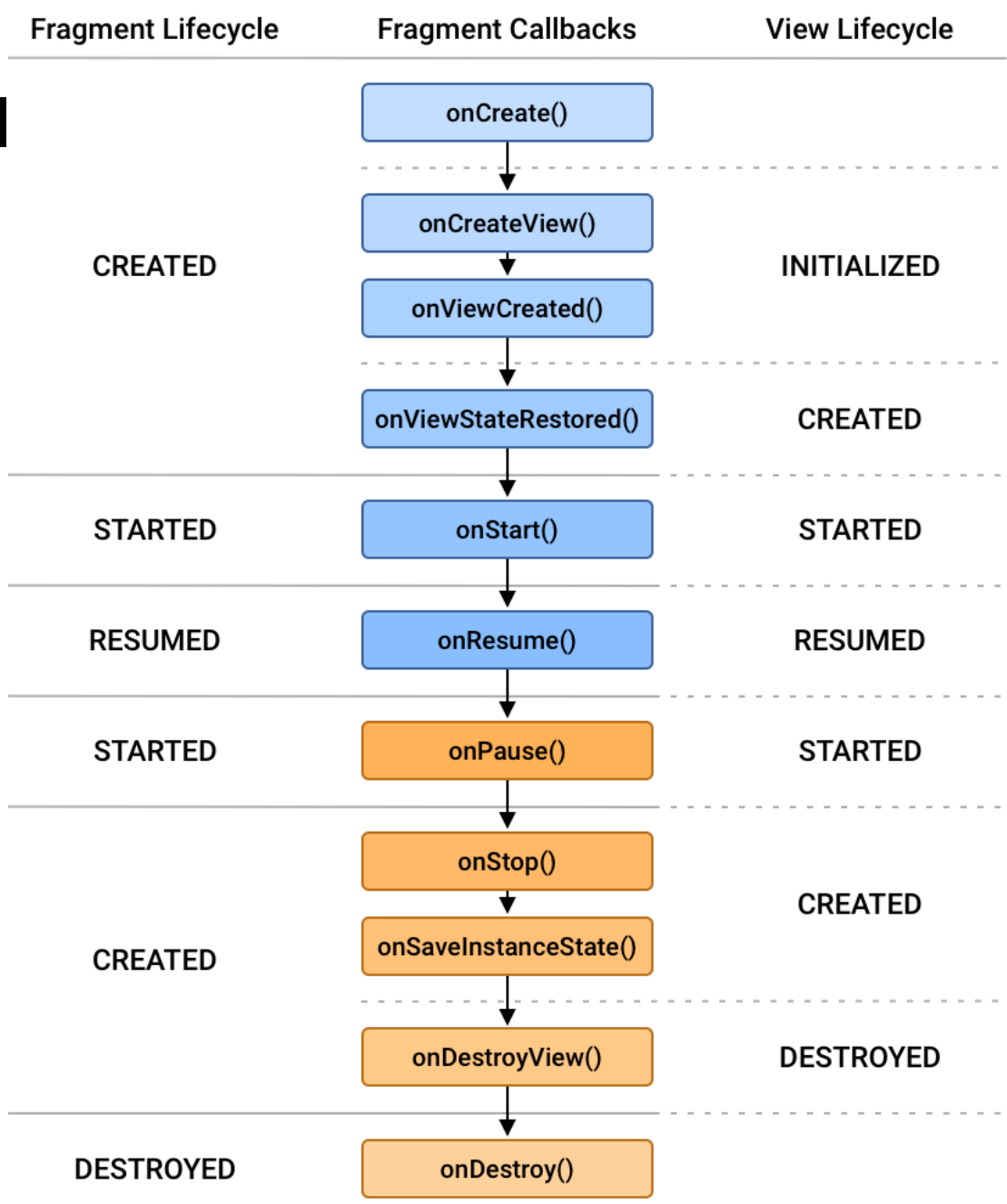
Жизненный цикл Fragment

При перемещении вверх по состояниям жизненного цикла фрагмент сначала вызывает соответствующий обратный вызов жизненного цикла для своего нового состояния. После завершения этого обратного вызова соответствующий Lifecycle.Event отправляется наблюдателям посредством Lifecycle фрагмента, за которым следует Lifecycle представления фрагмента, если он был создан.

Жизненный цикл Fragment

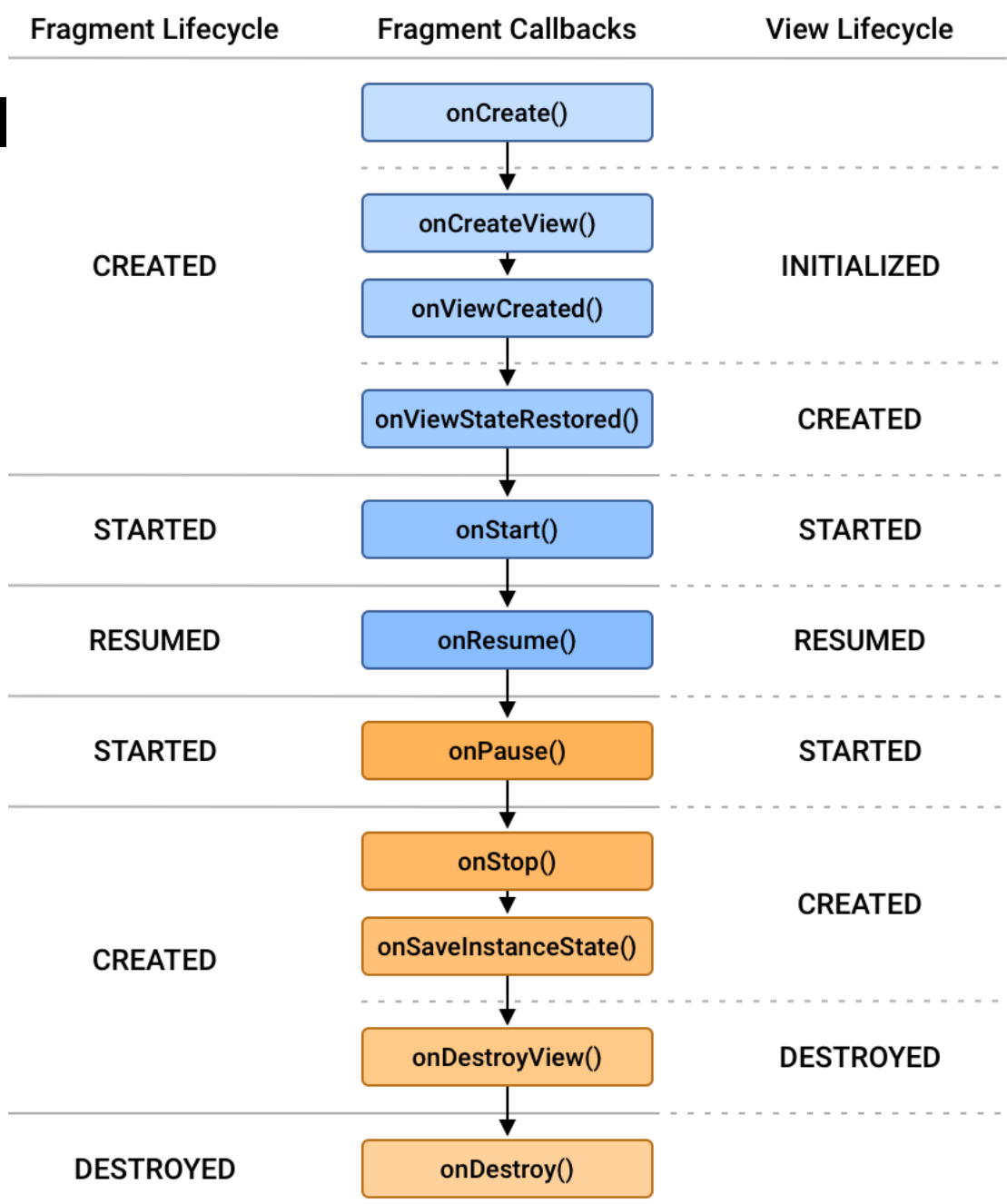
Когда ваш фрагмент достигает состояния CREATED, он добавляется в FragmentManager и уже вызван метод onAttach ().

Это подходящее место для восстановления любого сохраненного состояния, связанного с самим фрагментом, через его SavedStateRegistry. Обратите внимание, что представление фрагмента не было создано в это время, и любое состояние, связанное с представлением фрагмента, должно быть восстановлено только после того, как представление было создано.



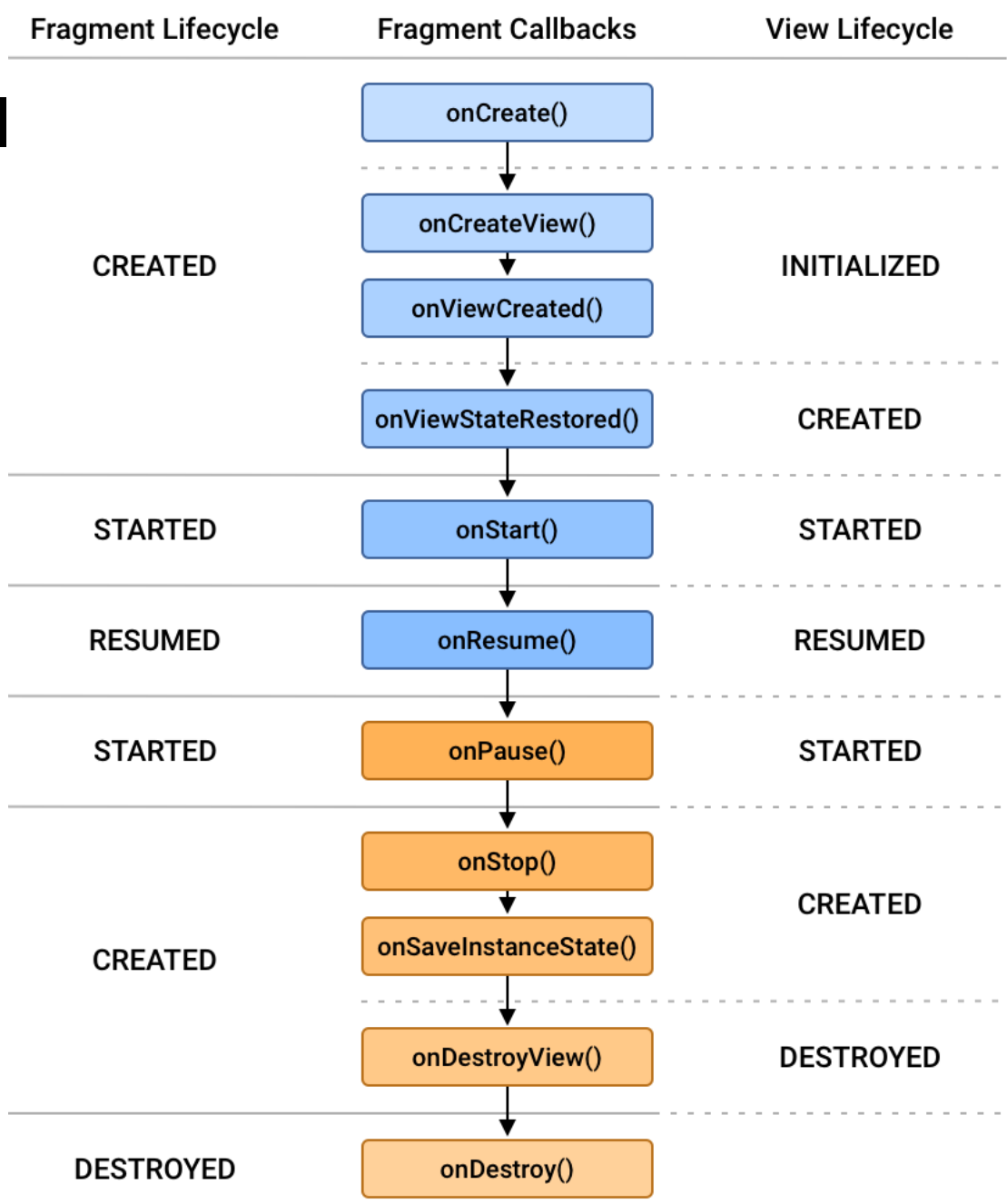
Жизненный цикл Fragment

Этот переход вызывает обратный вызов onCreate (). Обратный вызов также получает аргумент Bundle savedInstanceState, содержащий любое состояние, ранее сохраненное onSaveInstanceState (). Обратите внимание, что saveInstanceState имеет null при первом создании фрагмента, но оно всегда не равно null для последующих воссозданий, даже если вы не переопределяете onSaveInstanceState ().



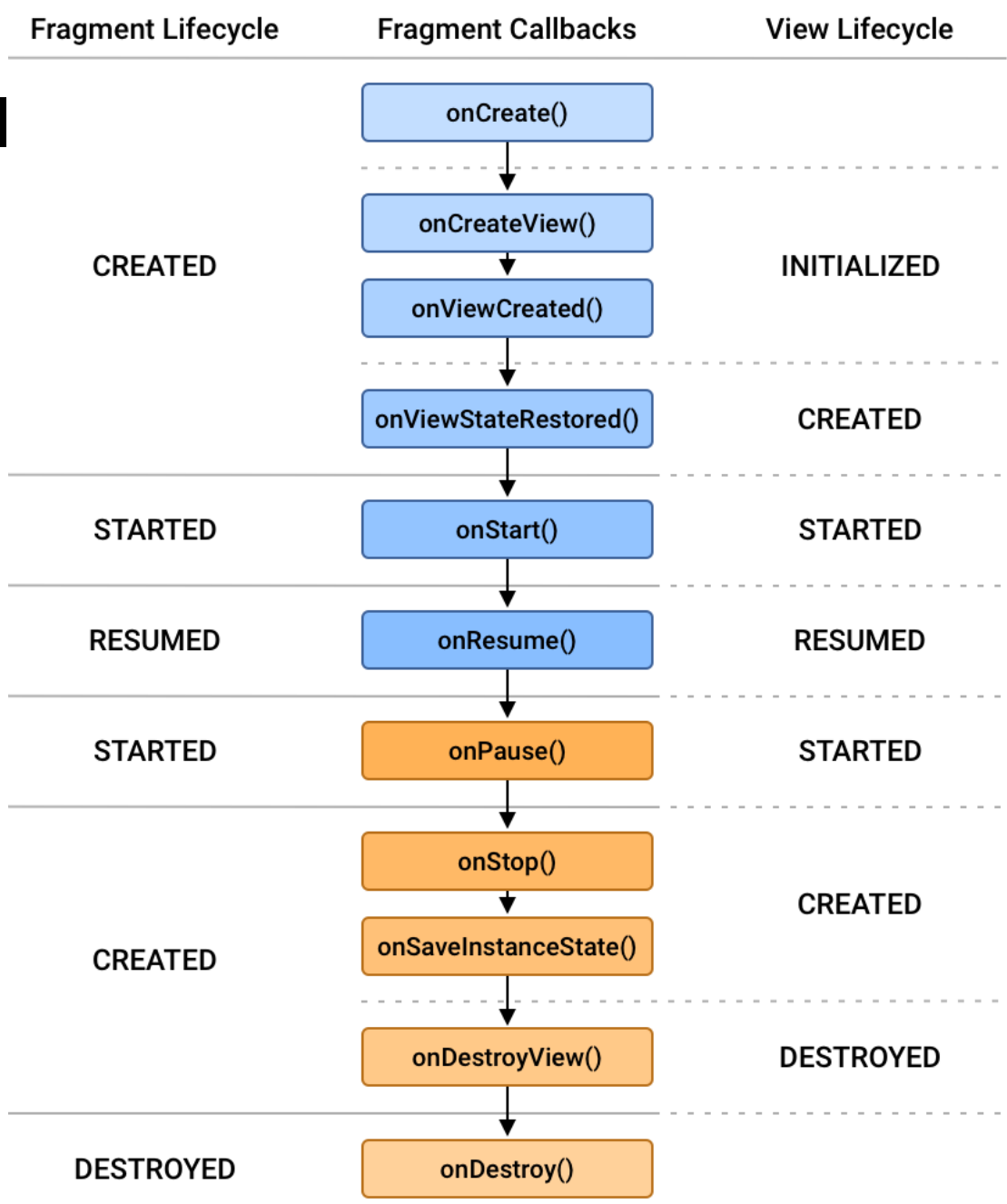
Жизненный цикл Fragment

Жизненный цикл представления фрагмента создается только тогда, когда ваш фрагмент предоставляет допустимый экземпляр представления. В большинстве случаев вы можете использовать конструкторы фрагментов, которые принимают @LayoutId, который автоматически растягивает представление в нужное время. Вы также можете переопределить onCreateView () для программного расширения или создания представления для вашего фрагмента.



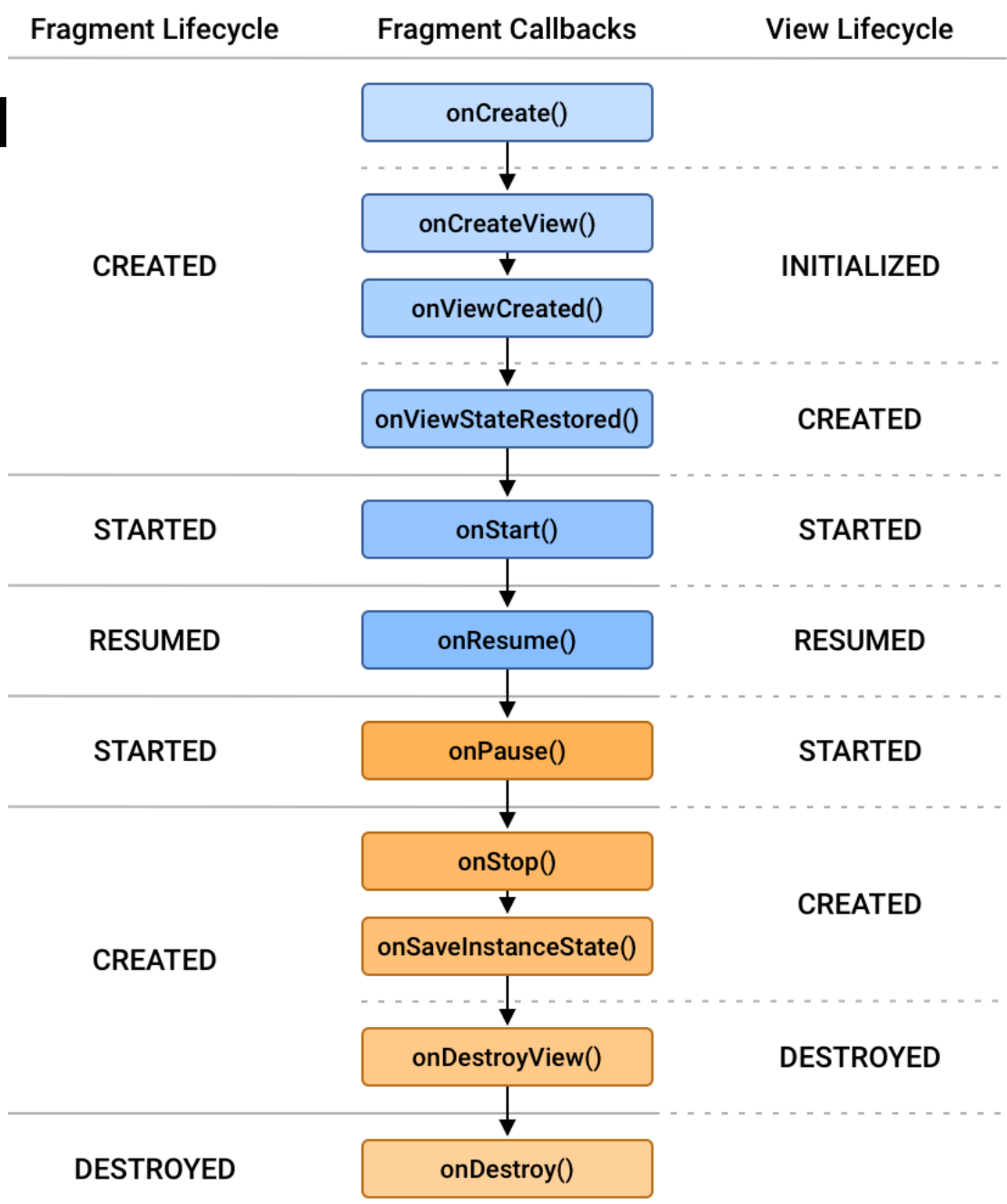
Жизненный цикл Fragment

Если и только если ваше представление фрагмента создается с ненулевым представлением, это представление устанавливается для фрагмента и может быть получено с помощью getView (). Затем getViewLifecycleOwnerLiveData () обновляется новым INITIALIZED LifecycleOwner, соответствующим представлению фрагмента. Обратный вызов жизненного цикла onViewCreated () также вызывается в это время.



Жизненный цикл Fragment

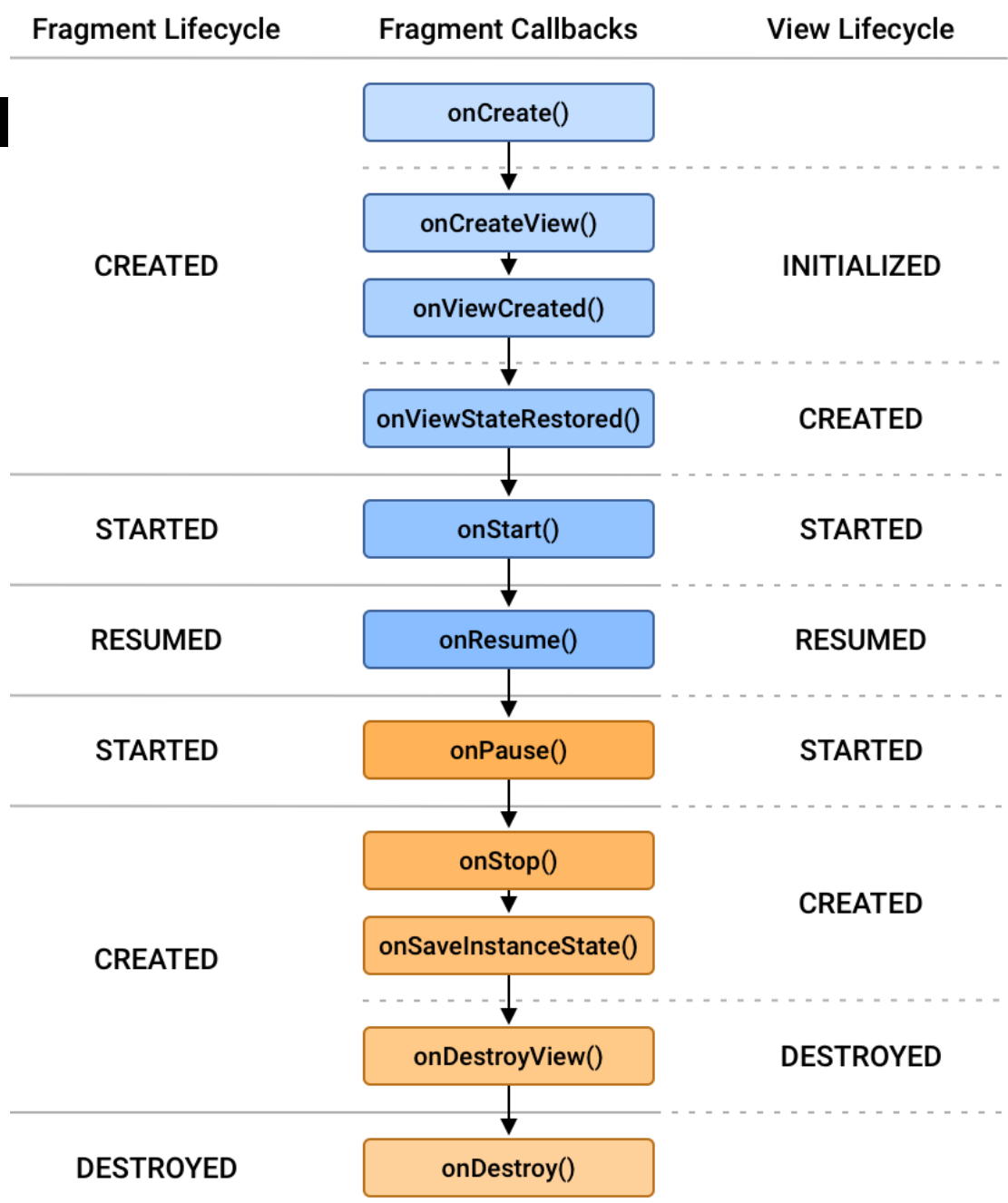
Это подходящее место для настройки начального состояния вашего представления, для начала наблюдения за экземплярами LiveData, чьи обратные вызовы обновляют представление фрагмента, и для настройки адаптеров для любых экземпляров RecyclerView или ViewPager2 в представлении вашего фрагмента.



Жизненный цикл Fragment

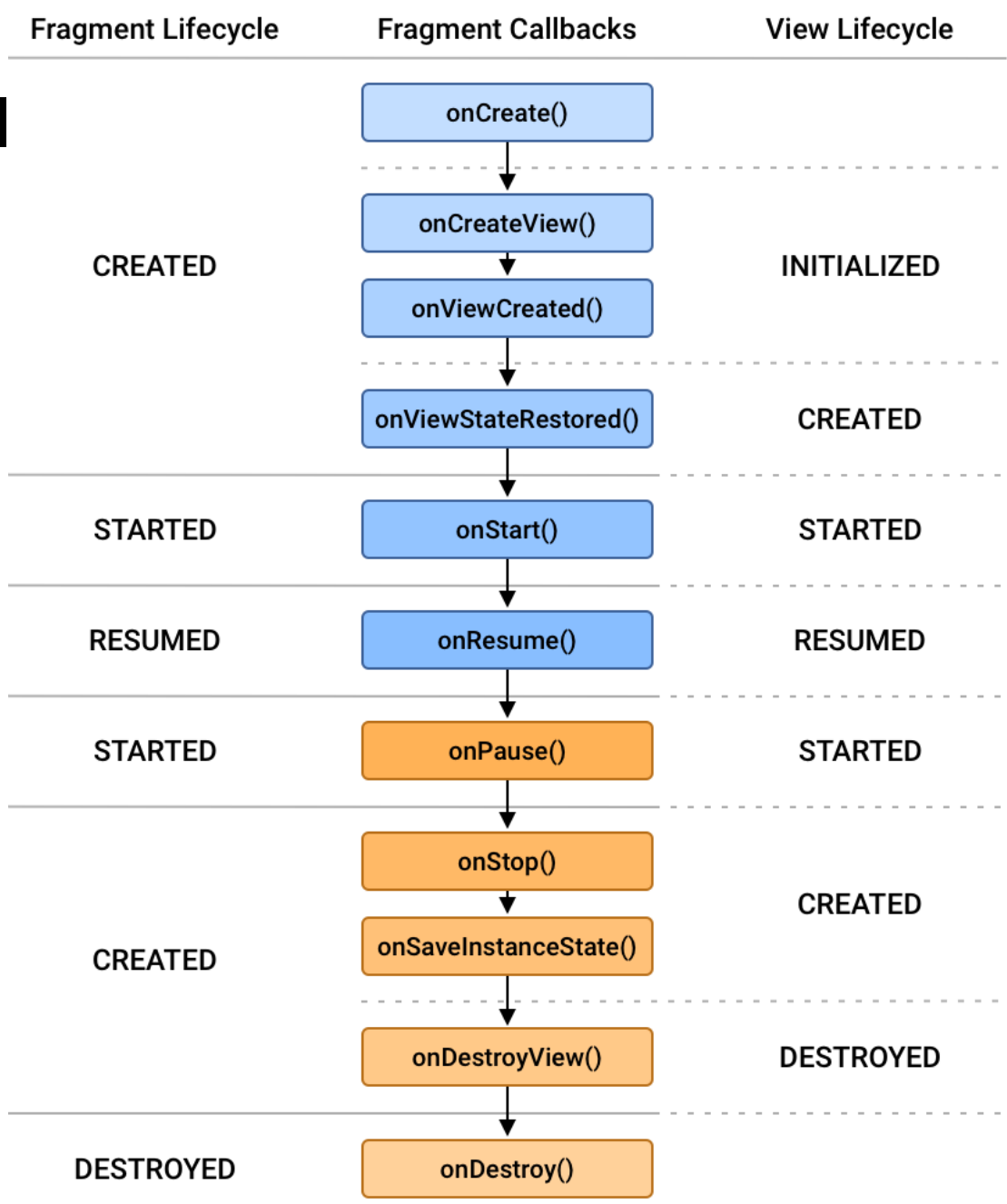
После создания представления фрагмента восстанавливается предыдущее состояние представления, если оно есть, и жизненный цикл представления затем переводится в состояние CREATED. Владелец жизненного цикла представления также отправляет своим наблюдателям событие ON_CREATE. Здесь вы должны восстановить любое дополнительное состояние, связанное с видом фрагмента.

Этот переход также вызывает обратный вызов onViewStateRestored ().



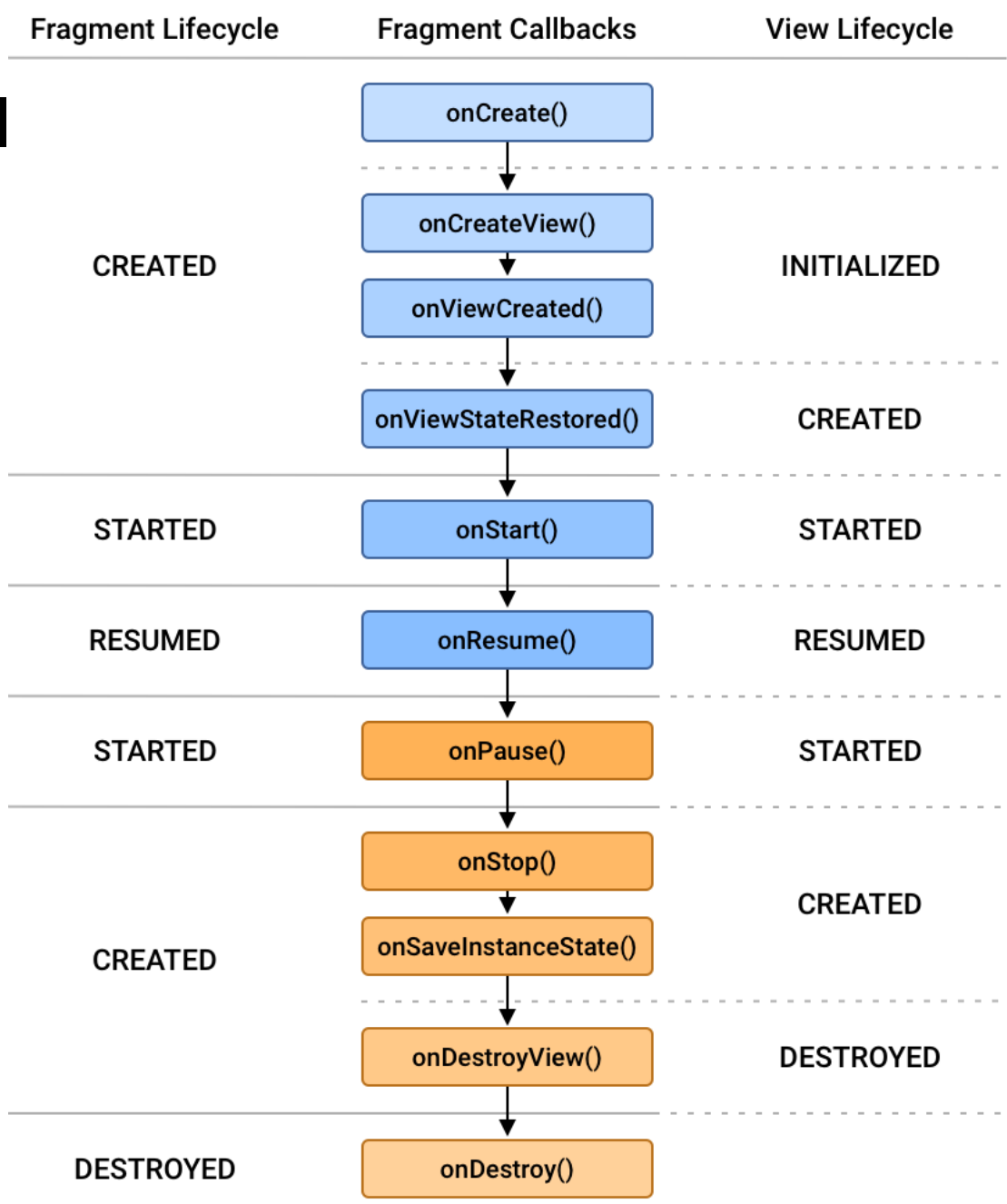
Жизненный цикл Fragment

Настоятельно рекомендуется привязать компоненты с учетом жизненного цикла к состоянию STARTED фрагмента, так как это состояние гарантирует, что представление фрагмента доступно, если оно было создано, и что безопасно выполнить FragmentTransaction для дочернего FragmentManager фрагмента. Если представление фрагмента не равно нулю, жизненный цикл представления фрагмента перемещается в STARTED сразу после того, как жизненный цикл фрагмента перемещается в STARTED.



Жизненный цикл Fragment

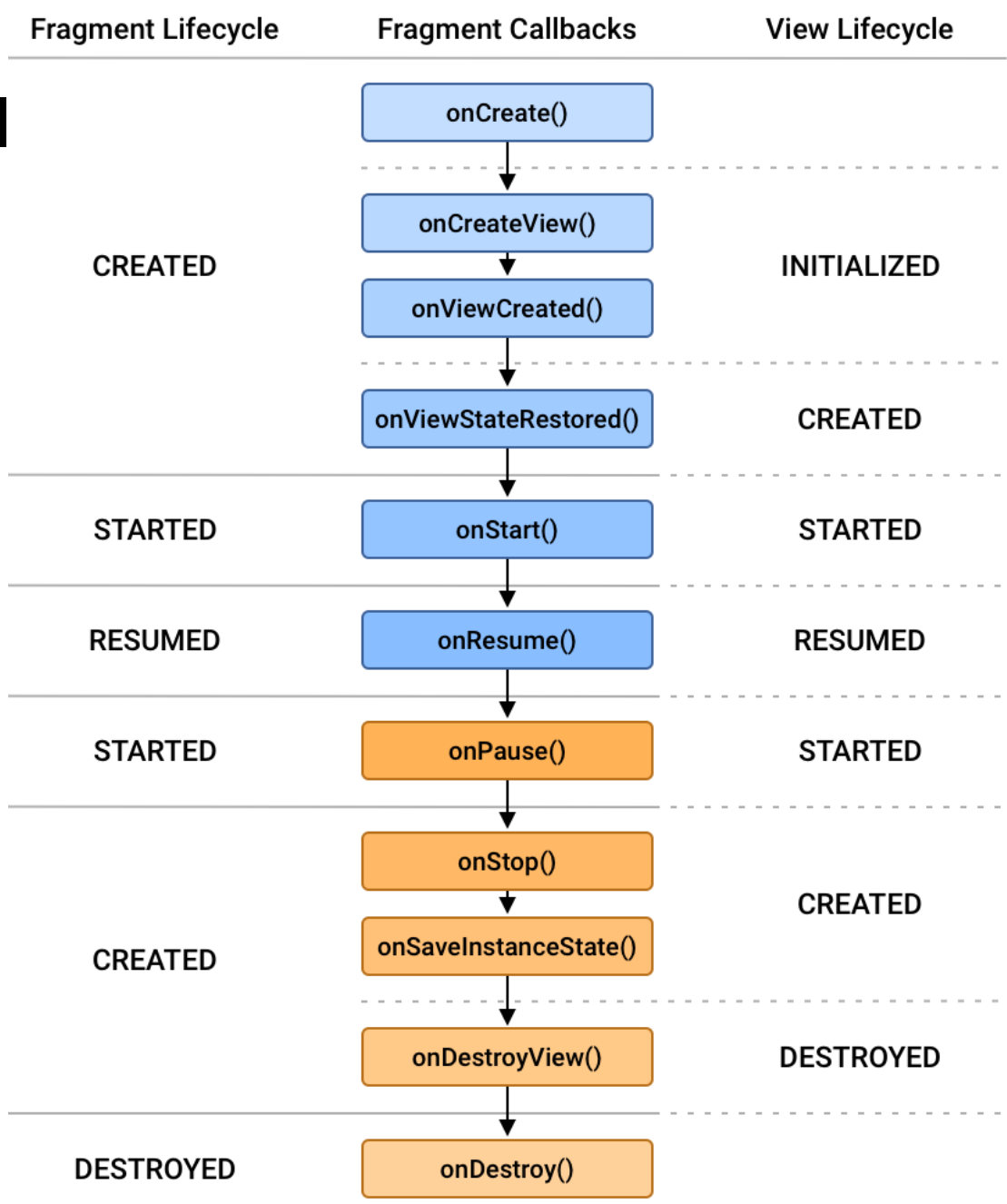
Когда фрагмент становится STARTED, вызывается обратный вызов onStart ().



Жизненный цикл Fragment

Когда фрагмент виден, все эффекты Animator и Transition завершены, и фрагмент готов для взаимодействия с пользователем. Жизненный цикл фрагмента переходит в состояние RESUMED, и вызывается обратный вызов onResume ().

Переход к RESUMED является подходящим сигналом, чтобы указать, что пользователь теперь может взаимодействовать с вашим фрагментом. Фрагменты, которые не возобновлены, не должны вручную устанавливать фокус на их представлениях или пытаться обрабатывать видимые методы ввода.

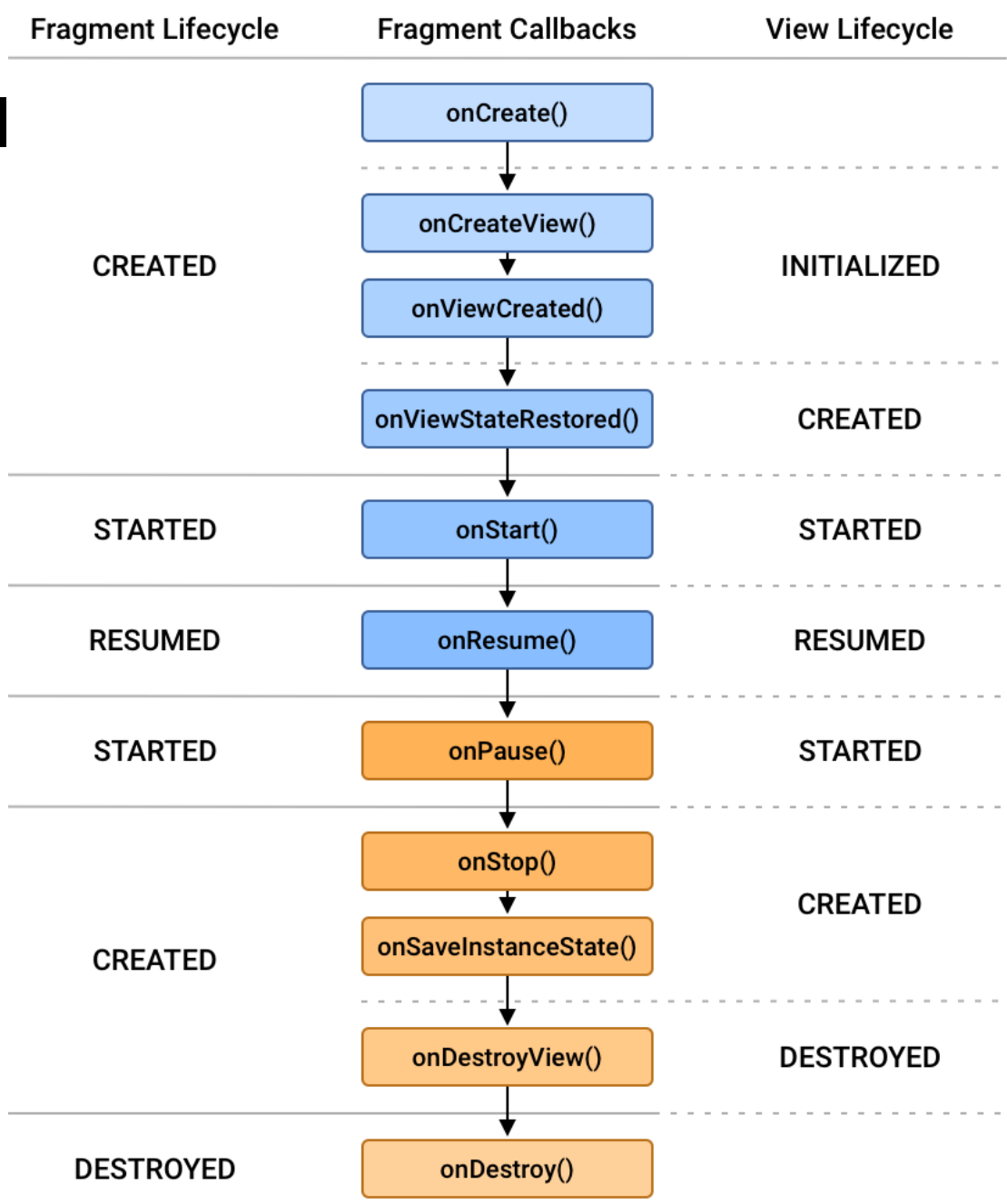


Жизненный цикл Fragment

Когда фрагмент перемещается вниз в более низкое состояние жизненного цикла, соответствующий Lifecycle.Event отправляется наблюдателям с помощью Lifecycle представления фрагмента, если он создан, за которым следует Lifecycle фрагмента. После того, как событие жизненного цикла фрагмента запущено, фрагмент вызывает соответствующий обратный вызов жизненного цикла.

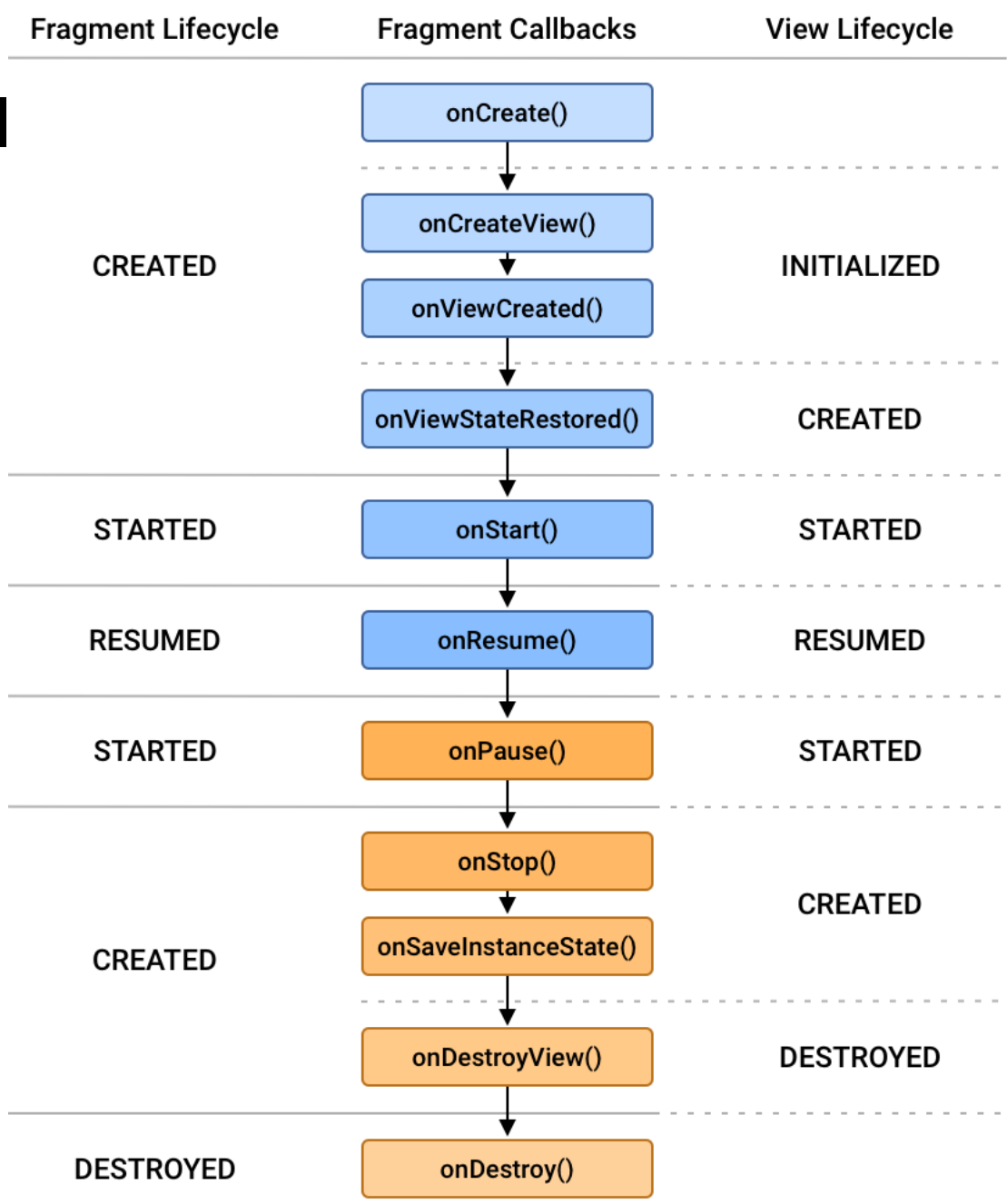
Жизненный цикл Fragment

Когда пользователь начинает покидать фрагмент, и пока фрагмент все еще виден, жизненные циклы для фрагмента и для его представления возвращаются в состояние STARTED и отправляют своим наблюдателям событие ON_PAUSE. Затем фрагмент вызывает обратный вызов onPause ().



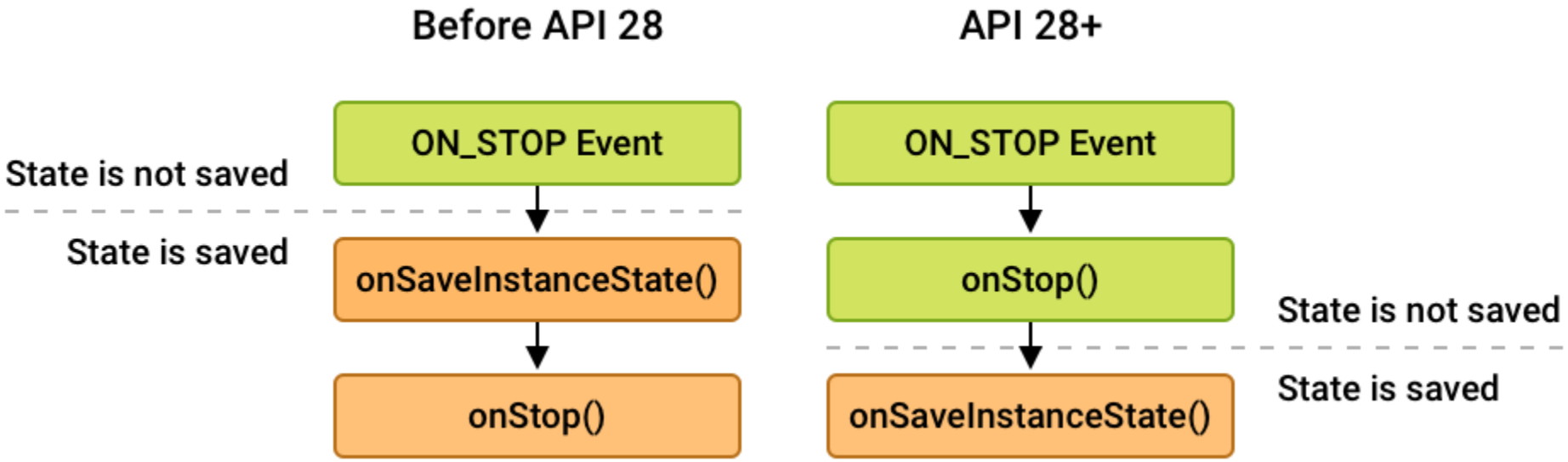
Жизненный цикл Fragment

Как только фрагмент больше не виден, жизненные циклы для фрагмента и для его представления переводятся в состояние CREATED и отправляют своим наблюдателям событие ON_STOP. Этот переход состояния запускается не только остановкой родительского activity или фрагмента, но и сохранением состояния родительским activity или фрагментом. Такое поведение гарантирует, что событие ON_STOP вызывается до сохранения состояния фрагмента. Это делает событие ON_STOP последней точкой, в которой можно безопасно выполнить FragmentTransaction для дочернего FragmentManager.



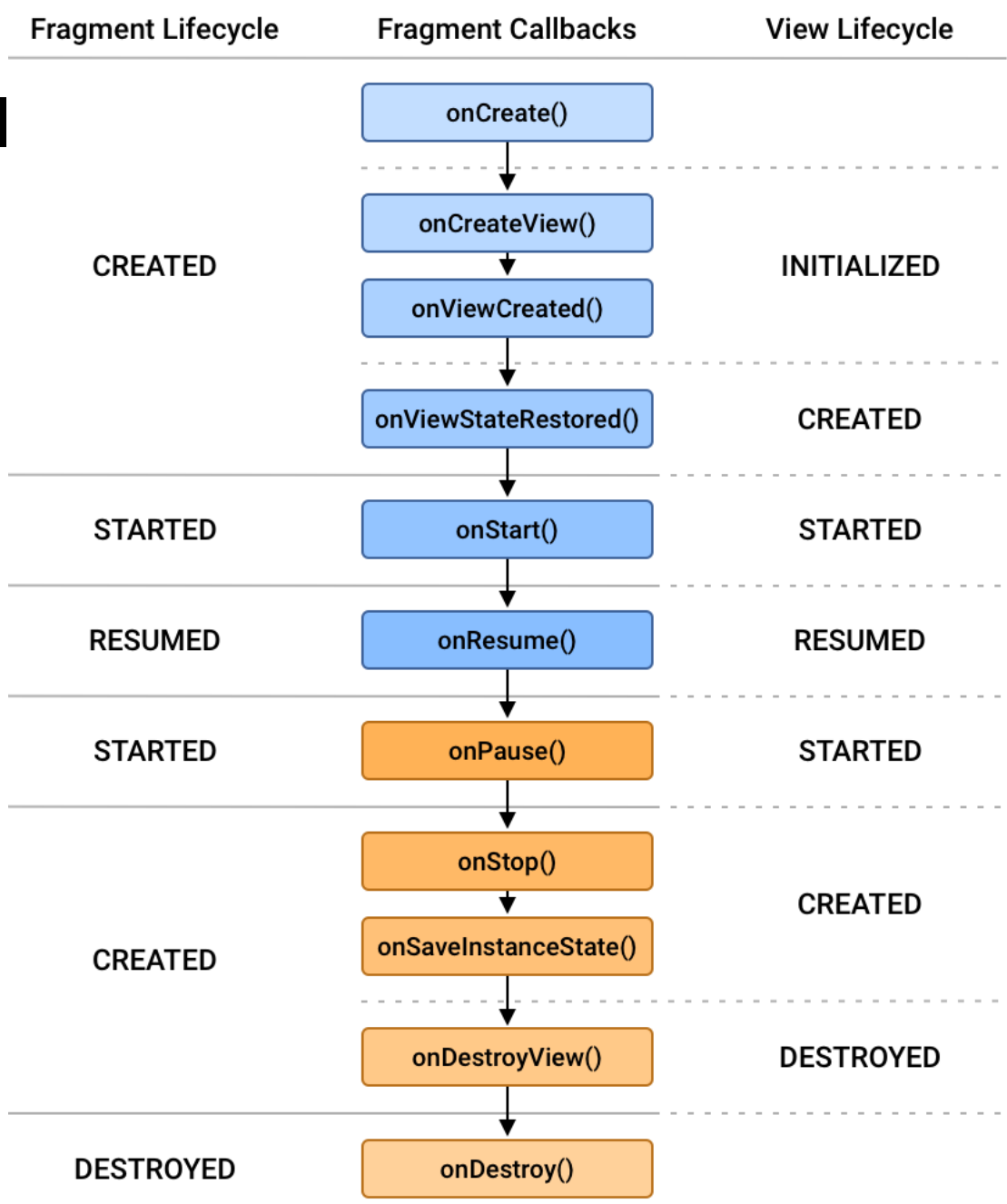
Жизненный цикл Fragment

Как показано на рисунке, порядок обратного вызова onStop () и сохранение состояния с помощью onSaveInstanceState () различаются в зависимости от уровня API. Для всех уровней API до API 28 onSaveInstanceState () вызывается перед onStop (). Для уровней API 28 и выше порядок вызовов обратный.



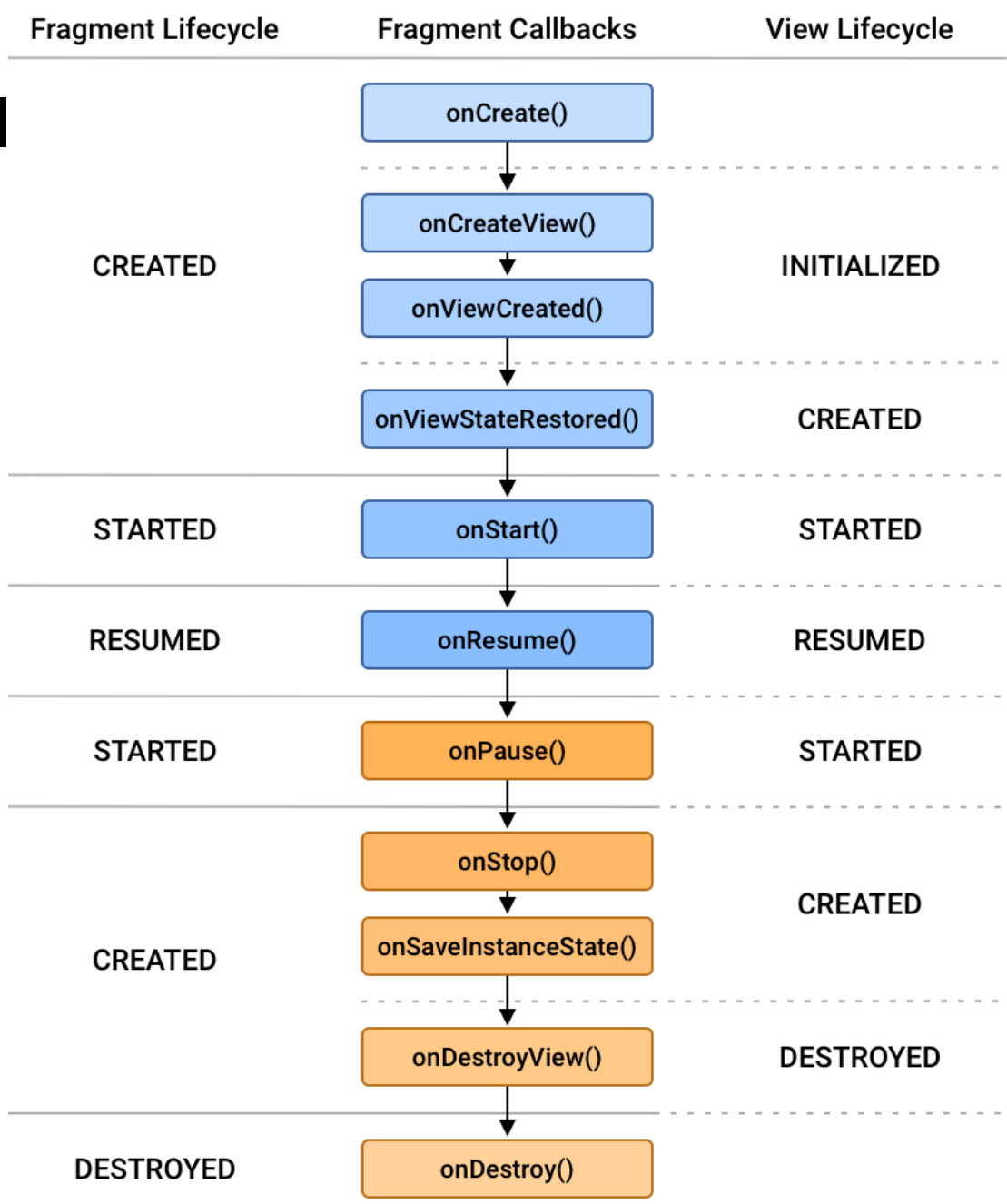
Жизненный цикл Fragment

После завершения всех анимаций выхода и переходов и отделения представления фрагмента от окна, жизненный цикл представления фрагмента переводится в состояние DESTROYED и передает своим наблюдателям событие ON_DESTROY. Затем фрагмент вызывает обратный вызов onDestroyView (). На этом этапе представление фрагмента достигло конца своего жизненного цикла, и getViewLifecycleOwnerLiveData () возвращает нулевое значение.



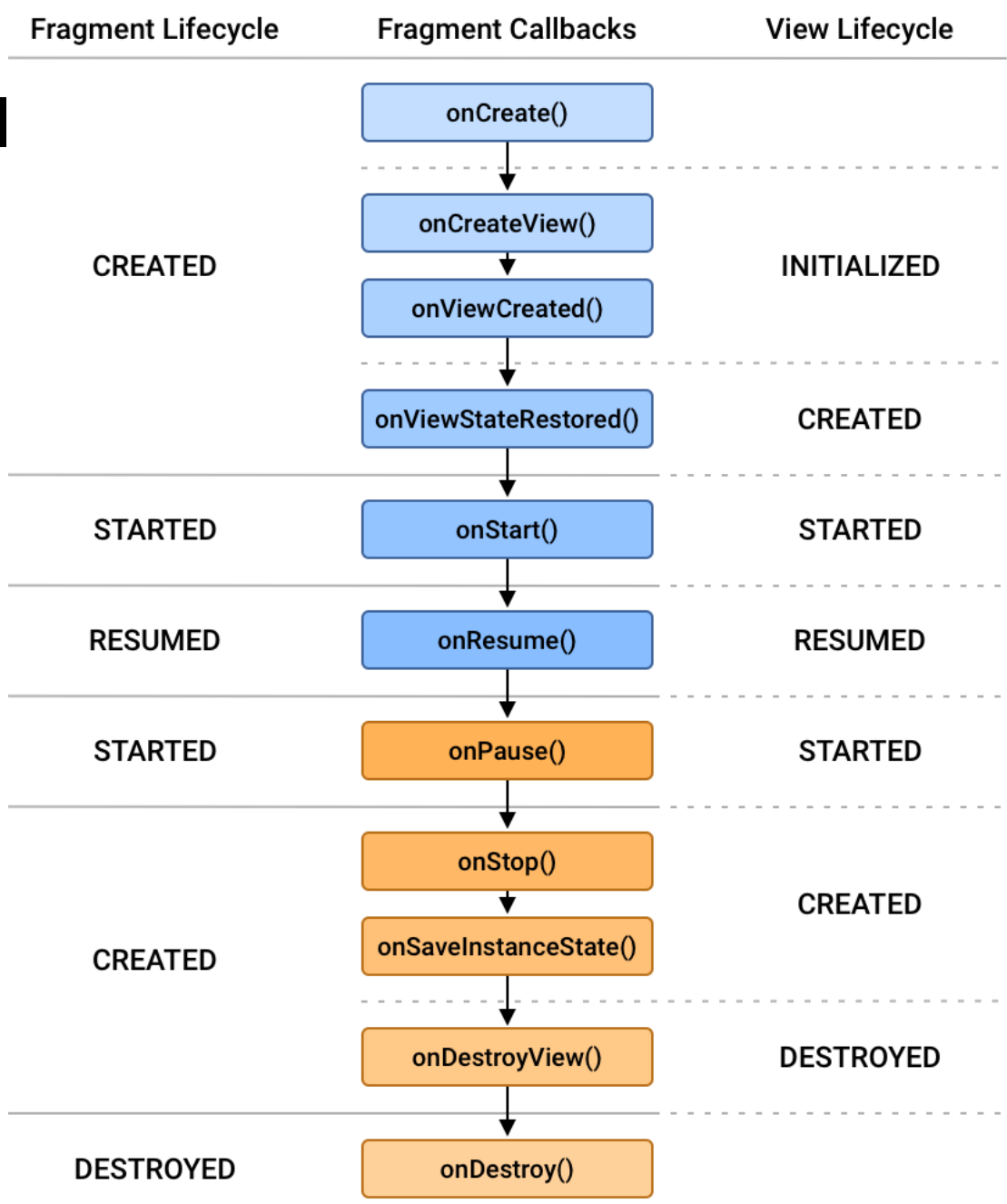
Жизненный цикл Fragment

На этом этапе все ссылки на представление фрагмента должны быть удалены, что позволит сборщику мусора удалить представления фрагмента.



Жизненный цикл Fragment

Если фрагмент удален или если FragmentManager уничтожен, жизненный цикл фрагмента переводится в состояние DESTROYED и отправляет событие ON_DESTROY своим наблюдателям. Затем фрагмент вызывает обратный вызов onDestroy (). На этом этапе жизненный цикл фрагмента подошел к концу.



Жизненный цикл Fragment

Различные версии ОС Android могут повлиять на состояние вашего фрагмента. Чтобы гарантировать сохранение состояния пользователя, платформа Android автоматически сохраняет и восстанавливает фрагменты и backstack. Следовательно, вам необходимо убедиться, что любые данные в вашем фрагменте также сохраняются и восстанавливаются.

Жизненный цикл Fragment

В следующей таблице описаны операции, которые приводят к потере состояния фрагмента, а также указано, сохраняются ли различные типы состояния при этих изменениях. Типы состояний, указанные в таблице, следующие:

- Variables: локальные переменные во фрагменте.
- View State представления: любые данные, принадлежащие одному или нескольким представлениям во фрагменте.
- SavedState: данные, присущие этому экземпляру фрагмента, которые должны быть сохранены в onSaveInstanceState ().
- NonConfig: данные, полученные из внешнего источника, такого как сервер или локальный репозиторий, или данные, созданные пользователем, которые отправляются на сервер после фиксации.

Жизненный цикл Fragment

Часто Variables обрабатываются так же, как SavedState, но в следующей таблице проводится различие между ними, чтобы продемонстрировать влияние различных операций на каждую из них.

Жизненный цикл Fragment

* Состояние NonConfig может быть сохранено после смерти процесса с помощью модуля Saved State для ViewModel.

Operation	Variables	View State	SavedState	NonConfig
Added to back stack	✓	✓	x	✓
Config Change	x	✓	✓	✓
Process Death/Recreation	x	✓	✓	✓*
Removed not added to back stack	x	x	x	x
Host finished	x	x	x	x

Жизненный цикл Fragment

View state

Представления отвечают за управление своим состоянием. Например, когда представление принимает вводимые пользователем данные, оно отвечает за сохранение и восстановление этих вводимых данных для обработки изменений конфигурации. Все представления, предоставляемые платформой Android, имеют собственную реализацию onSaveInstanceState () и onRestoreInstanceState (), поэтому вам не нужно управлять состоянием представления в вашем фрагменте.

Жизненный цикл Fragment

SavedState

Ваш фрагмент отвечает за управление небольшими объемами динамического состояния, которые являются неотъемлемой частью его функций. Вы можете сохранить легко сериализованные данные с помощью Fragment.onSaveInstanceState (Bundle). Подобно Activity.onSaveInstanceState (Bundle), данные, которые вы помещаете в пакет, сохраняются посредством изменений конфигурации и обработки смерти и воссоздания и доступны в onCreate (Bundle), onCreateView (LayoutInflater, ViewGroup, Bundle) и onViewCreated (View , Bundle) методах.

Жизненный цикл Fragment

SavedState

Примечание: onSaveInstanceState (Bundle) вызывается только тогда, когда host-activity фрагмента вызывает собственный onSaveInstanceState (Bundle).

Примечание: при использовании ViewModel вы можете сохранять состояние непосредственно в ViewModel с помощью SavedStateHandle.

Жизненный цикл Fragment

NonConfig

Данные NonConfig следует размещать вне вашего фрагмента, например, в ViewModel.

Класс ViewModel по своей сути позволяет данным сохраняться при изменениях конфигурации, таких как поворот экрана, и остается в памяти, когда фрагмент помещается в backstack. Добавление модуля SavedState в вашу ViewModel позволяет ViewModel сохранять простое состояние через смерть и восстановление процесса.